

CUDA-L1: Improving CUDA Optimization via Contrastive Reinforcement Learning

Xiaoya Li, Albert Wang, Guoyin Wang, Jiwei Li and Chris Shum

Ornith Team

 github.com/ornith-ai/CUDA-L1

Abstract

The exponential growth in demand for GPU computing resources has created an urgent need for automated CUDA optimization strategies. While recent advances in LLMs show promise for code generation, current state-of-the-art models achieve low success rates in improving CUDA speed. In this paper, we introduce CUDA-L1, an automated reinforcement learning (RL) framework for CUDA optimization that employs a novel contrastive RL algorithm.

CUDA-L1 achieves significant performance improvements on the CUDA optimization task: trained on NVIDIA A100, it delivers an average speedup of $\times 3.12$ with a median speedup of $\times 1.42$ against default baselines over across all 250 CUDA kernels of KernelBench, with peak speedups reaching $\times 120$. In addition to the default baseline provided by KernelBench, CUDA-L1 demonstrates $\times 2.77$ over Torch Compile, $\times 2.88$ over Torch Compile with reduce overhead, and $\times 2.81$ over CUDA Graph implementations. Furthermore, the model also demonstrates portability across GPU architectures, achieving average speedups of $\times 3.85$ (median $\times 1.32$) on H100, $\times 3.13$ (median $\times 1.31$) on L40, $\times 2.51$ (median $\times 1.18$) on RTX 3090, and $\times 2.38$ (median $\times 1.34$) on H20 despite being optimized specifically for A100.

Beyond these benchmark results, CUDA-L1 demonstrates several properties: CUDA-L1 1) discovers a variety of CUDA optimization techniques and learns to combine them strategically to achieve optimal performance; 2) uncovers fundamental principles of CUDA optimization, such as the multiplicative nature of optimizations; 3) identifies non-obvious performance bottlenecks and rejects seemingly beneficial optimizations that actually harm performance. The capabilities demonstrate that, RL can transform an initially poor-performing LLM into an effective CUDA optimizer through speedup-based reward signals alone, without human expertise or domain knowledge. In this process, it identifies CUDA optimization patterns, discovers new techniques, synthesizes them to achieve speedups, and more importantly, extends the acquired reasoning abilities to new kernels. This paradigm opens possibilities for automated optimization of CUDA operations, and holds promise to substantially promote GPU efficiency and alleviate the rising pressure on GPU computing resources. ¹

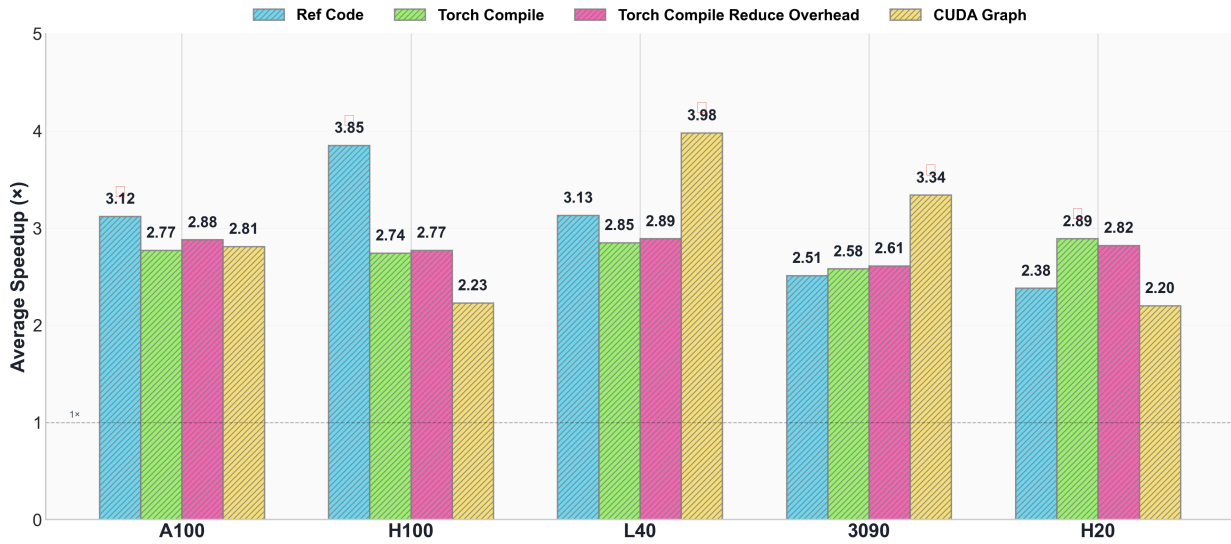


Figure 1: Average speedup across different optimization configurations on 5 types of GPU architectures.

¹This work is accepted by ICLR 2026.

 Email: {xiaoya_li, albert_wang, jiwei_li, chris_shum}@ornith.ai

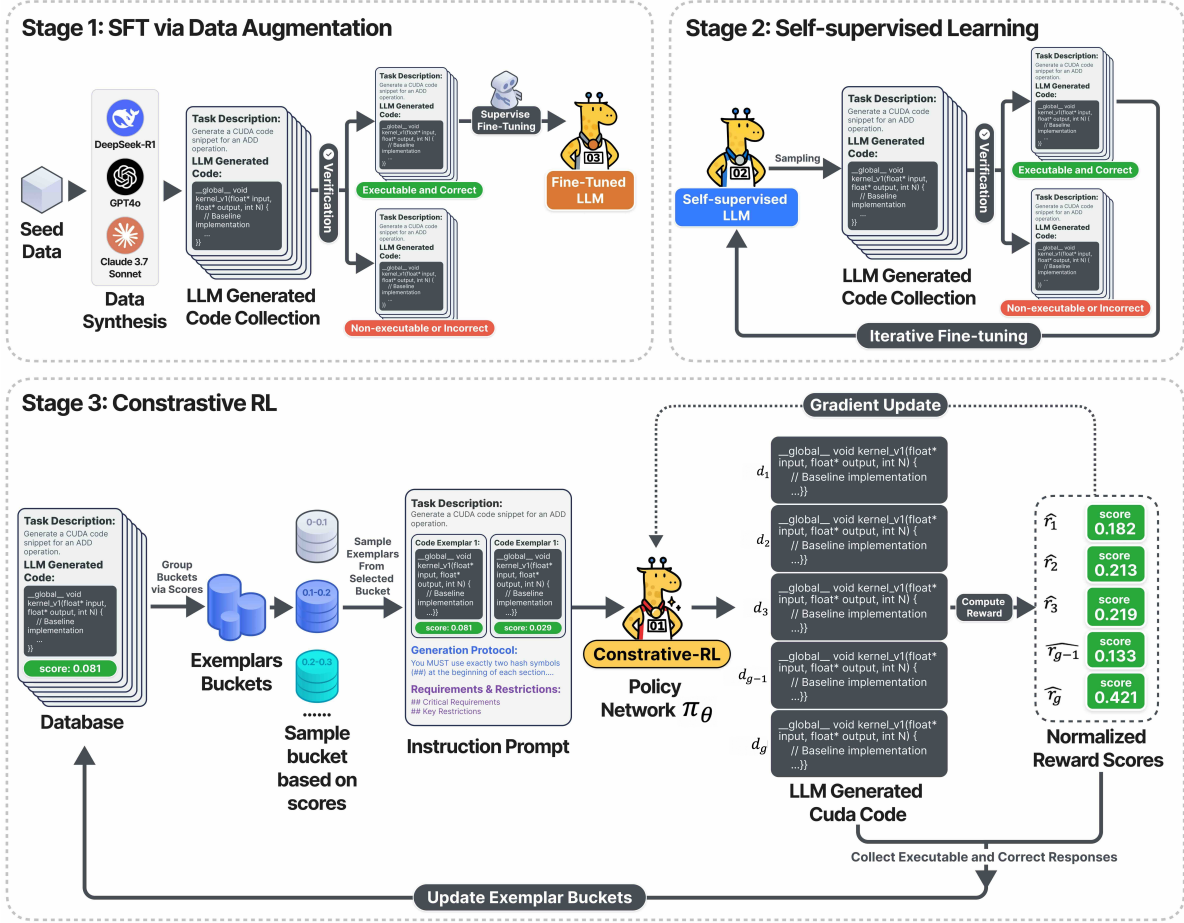


Figure 2: Overview of the CUDA-L1 training pipeline. The approach consists of three progressive stages: (1) **Stage 1: Supervised Fine-tuning with Data Augmentation** – We augment the training dataset with CUDA code variants generated by LLMs and fine-tune the base model on executable and correct implementations to establish foundational CUDA knowledge. (2) **Stage 2: Self-supervised Learning** – The model iteratively generates CUDA kernels, validates their correctness and executability, and trains on successfully validated examples, enabling autonomous improvement without human supervision. (3) **Stage 3: Contrastive Reinforcement Learning** – We employ contrastive learning with execution-time rewards, training the model to distinguish between faster and slower CUDA implementations, ultimately optimizing for superior performance.

1 Introduction

The exponential growth in demand for GPU computing resources, driven primarily by the rapid advancement and deployment of Large Language Models (LLMs), has created an urgent need for highly efficient CUDA optimization strategies. Traditionally, CUDA optimization has been a highly manual and time-intensive process, where skilled engineers must meticulously analyze memory access patterns, experiment with different thread block configurations, and iteratively profile their code through extensive trial-and-error cycles.

Recent advances in LLMs [25, 24, 26, 7, 32, 9, 11, 15, 19], especially those powered with RL [10, 8, 27, 17], have demonstrated remarkable capabilities in code generation and algorithm design. RL-powered LLMs hold significant potential to revolutionize the CUDA optimization process: CUDA optimization provides a uniquely clear reward signal—execution speed—which could be directly leveraged to automatically train reinforcement learning models. By treating performance improvements as rewards, RL-powered LLMs could iteratively generate, test, and refine CUDA optimizations without human intervention. This approach would not only automate the labor-intensive optimization process, potentially saving countless engineering hours, but also opens possibilities for discovering novel speedup algorithms that may surpass human-designed solutions. Unlike human engineers who are constrained by existing knowledge and conventional approaches, these systems could explore unconventional optimization combinations and potentially discover counterintuitive strategies that deliver significant performance improvements, offering new possibilities for advancing GPU computing efficiency.

Despite the promise, current performance remains limited. State-of-the-art LLM models such as DeepSeek-R1 [8] and OpenAI-o1 [10] achieve low success rates in generating optimized CUDA code (only approximately 15% on KernelBench [20]), which is primarily due to the scarcity of CUDA code in training datasets. To address these limitations and unlock the potential of LLMs for automated CUDA optimization, in this work, we propose CUDA-L1, an LLM framework powered by contrastive reinforcement learning for CUDA optimization. CUDA-L1 is a pipelined framework, the core of which is a newly-designed contrastive RL framework.

Different from previous RL models [31, 23, 22], contrastive RL performs comparative analysis of previously generated CUDA variants alongside their execution performance, enabling the model to improve through distinguishing between effective and ineffective optimization strategies. Contrastive-RL simultaneously optimizes the foundation model through gradient-based parameter updates while fulfilling the maximum potential from the current model through contrastive analysis from high-performance CUDA variants, creating a co-evolutionary dynamic that drives superior CUDA optimization performance.

CUDA-L1 delivers significant improvements on the CUDA optimization task: trained on NVIDIA A100, it achieves an **average** speedup of $\times 3.12$ (**median** $\times 1.42$) over the default baseline across all 250 KernelBench CUDA kernels, with maximum speedups reaching $\times 120$. In addition to the default baseline provided by KernelBench, CUDA-L1 demonstrates $\times 2.77$ over Torch Compile, $\times 2.88$ over Torch Compile with reduce overhead, $\times 2.81$ over CUDA Graph implementations. Furthermore, the CUDA codes optimized specifically for A100 demonstrate strong portability across GPU architectures, with similar optimization patterns observed across different baseline configurations: achieving average speedups of $\times 3.85$ (median $\times 1.32$) on H100, $\times 3.13$ (median $\times 1.31$) on L40, $\times 2.51$ (median $\times 1.18$) on RTX 3090, and $\times 2.38$ (median $\times 1.34$) on H20. Similar performance improvements over Torch Compile, Torch Compile with reduce overhead, CUDA Graph are consistently observed across all GPU types.

In addition to benchmark results, CUDA-L1 demonstrates several remarkable properties:

I) Automatic Discovery of Optimization Techniques: It automatically discovers a comprehensive range of optimization techniques, including both CUDA-specific optimizations—such as *memory layout optimization*, *operation fusion*, *loop unrolling*, and *memory coalescing*—and mathematical optimizations like *algebraic simplification*, *constant folding*, and *numerical approximation*. While some of these techniques are already widely adopted in the optimization community, others remain underutilized.

II) Optimal Combination Selection: Upon the discovery of these techniques, CUDA-L1 can identify the optimal combination of them to achieve maximum speedup for different CUDA tasks.

III) Uncovering Fundamental Principles: CUDA-L1 is able to uncover fundamental principles of CUDA optimization, such as the multiplicative nature of optimizations and how certain “gatekeeper” techniques must be applied first to unlock the effectiveness of others.

IV) Identifying Hidden Bottlenecks: It identifies non-obvious performance bottlenecks and rejects seemingly beneficial optimizations that actually harm performance.

Beyond, CUDA-L1 reveals a remarkable capability of RL in autonomous learning for CUDA optimization:

1. Even starting with a foundation model with poor CUDA optimization ability, by using code speedups as RL rewards and proper contrastive RL training techniques, we can still train an RL system capable of generating CUDA optimization codes with significant speedups.
2. Without human prior knowledge, RL systems can independently discover CUDA optimization techniques, learn to combine them strategically, and more importantly, extend the acquired CUDA reasoning abilities to unseen kernels. This capability unlocks the potential for a variety of automatic CUDA optimization tasks, e.g., kernel parameter tuning, memory access pattern optimization, and different hardware adaptations, offering substantial promises to enhance GPU utilization during this period of unprecedented computational demand.

diag(A) * B | Reference Code

```

1  class Model(nn.Module):
2      def forward(self, A, B):
3          # A: (N,) - 1D tensor of shape N
4          # B: (N, M) - 2D tensor of shape N x M
5          # torch.diag(A): (N, N) - creates
            diagonal matrix from A
6          # Result: (N, N) @ (N, M) = (N, M)
7          return torch.diag(A) @ B

```

diag(A) * B | CUDA-L1 Implementation (64x faster)

```

1  class Model(nn.Module):
2      def forward(self, A, B):
3          return A.unsqueeze(1) * B

```

A case from KernelBench (Level 1, Task 12), computing $\text{diag}(A) * B$. We present reference code and CUDA-L1 implementation. The CUDA-L1 implementation reduces complexity from $O(N^2 M)$ to $O(NM)$, achieving **64x** speedup by replacing full matrix multiplication with element-wise operations.

Another contribution of this work is the enrichment of the KernelBench dataset with CUDA Graph implementations.² We release these implementations to the community, providing substantially stronger baselines for performance comparison. Despite its capabilities, it is important to acknowledge the potential challenges and pitfalls in training RL systems for tasks including but not limited to CUDA kernel development. We discovered that RL can be remarkably adept at exploiting loopholes in reward systems rather than solving the intended problem. For example, in our experiments, RL discovered a vulnerability in the KernelBench evaluation: by creating additional CUDA streams in the generated code, it could manipulate the timing measurements. This exploitation resulted in a reported 18x speedup that was entirely artificial with the actual computation performance unchanged. Such reward hacking behaviors are particularly concerning because they often require careful human inspection to detect. This paper identifies these failure modes, and proposes practical methods for more robust reward mechanisms and training procedures.

2 CUDA-L1

2.1 Overview

Existing large language models [8, 32, 7] demonstrate significant limitations in generating executable and correct CUDA code with speedup, as reported in prior research [20]. This deficiency likely stems from the insufficient representation of CUDA code in the training datasets of these models. To address this fundamental gap, we introduce a three-stage pipelined training strategy for CUDA-L1, aiming to progressively enhance the model’s CUDA programming capabilities:

1. **Supervised fine-tuning via data augmentation**, which aims to expand the model’s exposure to CUDA patterns and programming constructs, with the primary goal of producing correct and executable CUDA code.
2. **Self-supervised learning** which focuses on enabling models to develop a deeper understanding of CUDA semantics and programming principles, primarily aiming to achieve significant improvements in executability and correctness, while providing moderate speedup gains.
3. **Contrastive reinforcement learning**, which is designed to significantly optimize code execution speed, with the goal of generating high-performance CUDA implementations that deliver substantial speedup.

Before we delve into the details of each stage, we provide key definitions adopted throughout the rest of this paper:

1. **Executability**: A CUDA code is executable if it successfully compiles, launches, and executes to completion within $1000\times$ the runtime of the reference implementation. Code exceeding this runtime threshold is considered unexecutable.³
2. **Correctness**: A CUDA code is correct if it produces equivalent outputs to the reference implementation across 1000 random test inputs.⁴
3. **Success**: A CUDA code is successful if it is both executable and correct.

2.2 SFT via Data Augmentation

In the SFT stage, we collect a dataset by using existing LLMs to generate CUDA code snippets and selecting successful one. This dataset is directly used to fine-tune the model. Throughout this paper, we use deepseek-v3-671B [15] as the model backbone.

Data Collection To expand the model’s exposure to CUDA patterns, we begin with data augmentation based on reference code from 250 tasks in KernelBench, which provides the official implementations used in PyTorch. To generate executable and correct CUDA code efficiently, we leverage six existing LLM models: GPT-4o, OpenAI-o1, DeepSeek-R1, DeepSeek V3, Llama 3.1-405B Instruct, and Claude 3.7. For each model, we construct prompts using the one-shot strategy, where the prompt contains the reference code (denoted by $q_i, i \in [1, 250]$) and asks the LLM to generate an alternative speedup implementation. We employ multiple models to maximize the diversity of successful CUDA code generation. The detailed prompt structure is provided in Table 1. For each of the six models, we iterate through all 250 tasks. Each task allows up to 20 trials and terminates early if we successfully collect 2 trials that are both executable and correct. Notably, some tasks may fail to produce any successful code across all trials. The successful code is denoted by $d_{i,j}$, where $j \in \{1, 2, \dots, n_i\}$, and n_i denotes the number of successful code snippets for the reference code q_i . Through this process, we collected 2,105 successful CUDA code snippets. Now we have collected the dataset $D = \{(q_i, \{d_{i,j}\}_{j=1}^{n_i})\}_i$.

²Found at <https://github.com/deepreinforce-ai/CUDA-L1>

³This threshold is reasonable since code with $1000\times$ slower performance contradicts our speedup optimization goals.

⁴Prior work uses only 5 random inputs, which we found insufficient for robust validation.

The collected dataset D is used to finetune the foundation model. The instruction to the model is the same as the prompt for dataset generation, where the reference code q_i is included in the instruction and the model is asked to generate an improved version. The model is trained to predict each token in $d_{i,j}$ given the instruction.

2.3 Self-supervised Learning

Now we are presented with the finetuned model after the SFT stage, where the model can potentially generate better CUDA code with higher success rates than the original model without finetuning. We wish to further improve the model’s ability to generate successful CUDA code by exposing it to more code snippets generated by itself.

We achieve this iteratively by sampling CUDA code from the model, evaluating it for executability and correctness, removing the unsuccessful trials and keeping the successful ones. Successful ones are batched and used to update the model parameters. Using the updated model, we repeat the process: generating code, evaluating it, and retraining the model. The psudo code for the algorithm is shown in Table 1.

The self-supervised learning strategy can be viewed as a special case of the REINFORCE algorithm [31], a typical policy gradient reinforcement learning method, where the reward is set to 1 for successful trials and 0 for unsuccessful trials, without applying any baseline. Interestingly, we find this adopted training strategy to be more stable than the REINFORCE variant with baseline applied. We conjecture that this stability arises because during the self-supervised learning stage, a significant proportion of generated instances remain unsuccessful. This approach avoids the potential instability caused by applying negative updates to unsuccessful samples when using a baseline.

It is worth noting that during the self-supervised learning stage, we focus exclusively on the executability and correctness of the generated code, without considering speed as a metric. This design choice reflects our primary objective of establishing reliable code generation before optimizing for performance.

Self-supervised Learning Algorithm

```

1: Initialize finetuned model  $M_0$  after SFT stage with parameters  $\theta_{\text{sft}}$ 
2: for  $i = 1$  to  $N_{\text{iterations}}$  do
3:   Generate batch of CUDA codes  $C_i = \{c_1, \dots, c_k\}$  using model  $M_{i-1}$ 
4:   Evaluate each  $c \in C_i$  for:
5:     1. Executability (compiles and runs)
6:     2. Correctness (produces expected output)
7:   Filter successful codes:  $C_i^{\text{success}} = \{c \in C_i | \text{executable} \wedge \text{correct}\}$ 
8:   if  $C_i^{\text{success}} \neq \emptyset$  then
9:     Compute gradient update  $\nabla\theta$  using  $C_i^{\text{success}}$ 
10:    Update model:  $\theta_i \leftarrow \theta_{i-1} + \eta \nabla\theta$ 
11:   else
12:      $\theta_i \leftarrow \theta_{i-1}$  (no update)
13:   end if
14: end for
15: return Final improved model  $M_N$ 

```

Table 1: Self-supervised learning for cuda optimization in Stage 2.

2.4 Contrastive Reinforcement Learning

Now we have a model capable of generating successful CUDA code at a reasonable success rate. Next, we aim to optimize for execution speed.

One straightforward approach is to apply existing reinforcement learning algorithms such as REINFORCE [31], GRPO [23], or PPO [22]. In this approach, we would ask the model to first perform chain-of-thought reasoning [29], then generate code, evaluate it, and use the evaluation score to update the model parameters. However, our experiments reveal that these methods perform poorly in this task. The issue is as follows: standard RL algorithms compute a scalar reward for each generated CUDA code sample. During training, this reward undergoes algorithm-specific processing (e.g., baseline subtraction in REINFORCE, advantage normalization in GRPO, importance sampling in PPO). The processed reward then serves as a loss weighting term for gradient updates, increasing the likelihood of high-reward sequences while decreasing the likelihood of low-reward sequences.

Critically, in this paradigm, the reward signal is used exclusively for parameter updates and is never provided as input to the LLM. Consequently, the LLM cannot directly reason about performance trade-offs during code generation.

Data Augmentation Prompt — Used in Supervised fine-tuning
Task for CUDA Optimization You are an expert in CUDA programming and GPU kernel optimization. Now you’re tasked with developing a high-performance cuda implementation of Softmax. The implementation must: • Produce identical results to the reference PyTorch implementation. • Demonstrate speed improvements on GPU. • Maintain stability for large input values.
Reference Implementation (exact copy) <pre> 1 import torch 2 import torch.nn as nn 3 4 class Model(nn.Module): 5 """ 6 Simple model that performs a Softmax activation. 7 """ 8 def __init__(self): 9 super(Model, self).__init__() 10 11 def forward(self, x: torch.Tensor) -> torch.Tensor: 12 """ 13 Applies Softmax activation to the input tensor. 14 Args: 15 x (torch.Tensor): Input tensor of shape (batch_size, num_features). 16 Returns: 17 torch.Tensor: Output tensor with Softmax applied, same shape as input. 18 """ 19 return torch.softmax(x, dim=1) 20 21 batch_size = 16 22 dim = 16384 23 24 def get_inputs(): 25 x = torch.randn(batch_size, dim) 26 return [x] 27 28 def get_init_inputs(): 29 return [] # No special initialization inputs needed </pre>

Table 2: Prompt illustration for data augmentation in Section 2.2. For each KernelBench task (softmax shown here for illustration), the prompt is fed to each of six LLM models—GPT-4o, OpenAI-o1, DeepSeek-R1, DeepSeek V3, Llama 3.1-405B Instruct, and Claude 3.7 Sonnet—to generate alternative CUDA implementations.

To address this limitation, we propose incorporating reward information directly into the reasoning process by embedding performance feedback within the input prompt. Specifically, we present the model with multiple code variants alongside their corresponding speedup scores. Rather than simply generating code, the LLM is trained to first conduct comparative analysis of why certain implementations achieve superior performance, then synthesize improved solutions based on these insights. Each generated code sample undergoes evaluation to obtain a performance score, which serves dual purposes in our training framework:

1. **Immediate Parameter Updates:** The score functions as a reward signal for gradient-based parameter optimization, directly updating the model weights.
2. **Future Prompt Construction:** The scored code sample becomes part of the exemplar set for subsequent training iterations, enriching the contrastive learning dataset.

This dual-utilization strategy enables iterative optimization across two complementary dimensions:

1. **Foundation Model Enhancement:** Parameter updates progressively improve the model’s fundamental understanding and capabilities for CUDA optimization tasks, expanding its representational capacity.
2. **Fixed-Parameter Solution Optimization:** The contrastive approach seeks to extract the maximum potential from the current model’s parameters by leveraging comparative analysis of high-quality exemplars.

These two optimization processes operate synergistically: enhanced foundation models enable more accurate contrastive reasoning, while improved reasoning strategies provide higher-quality training signals for parameter updates of foundation models. This co-evolutionary dynamic drives convergence toward optimal performance. We term this approach contrastive reinforcement learning (contrastive-RL for short).

It is worth noting that this co-evolving optimization paradigm can be found in many machine learning frameworks, including the EM algorithm [1], where the E-step optimizes latent variable assignments given fixed parameters while the M-step updates parameters given fixed assignments; Variational inference [3], which alternately optimizes the variational parameters to approximate the posterior distribution and updating model parameters to maximize the evidence lower bound; Actor-critic methods [12] in reinforcement learning similarly alternate between policy evaluation (critic update) and policy improvement (actor update).

2.4.1 Contrastive-RL’s Advantages over Evolutionary LLM Approaches

Contrastive-RL draws inspiration from a broad range of literature, including evolutionary algorithms [2] and their applications to LLMs [16, 21, 18, 28], where multiple solution instances with associated fitness scores are presented to LLMs to analyze performance patterns and generate improved solutions. However, Contrastive-RL improves evolutionary LLM approaches in several critical aspects:

Model Adaptation vs. Fixed-Model Reasoning: Contrastive-RL employs gradient-based parameter updates to continuously enhance model capabilities, whereas evolutionary LLM approaches rely exclusively on in-context learning with static parameters. This fundamental architectural difference endows Contrastive-RL with substantially greater representational capacity and task adaptability. Evolutionary LLM methods are fundamentally limited by the frozen foundation model’s initial knowledge and reasoning abilities, while Contrastive-RL progressively refines the model’s domain-specific expertise through iterative parameter optimization. From this perspective, evolutionary LLM approaches can be viewed as a degenerate case of Contrastive-RL that implements only the Fixed-Parameter Solution Optimization component while omitting the Foundation Model Enhancement mechanism. This theoretical relationship explains why Contrastive-RL consistently outperforms evolutionary approaches: it leverages both optimization dimensions simultaneously rather than constraining itself to a single fixed-capacity search space.

Scalability and Generalization: Contrastive-RL demonstrates superior scalability by training a single specialized model capable of handling diverse CUDA programming tasks and generating various types of optimized code. In contrast, evolutionary LLM approaches typically require separate optimization processes for each distinct task or domain, limiting their practical applicability and computational efficiency.

2.4.2 Prompt Construction

Here we describe the construction of prompts provided to the LLM. The prompt provided to the LLM during Contrastive-RL training comprises the following structured components:

- **Task Description:** A detailed description of the computational problem, including input/output specifications, performance requirements, and optimization objectives.
- **Previous Cuda Codes with Scores:** Previously generated CUDA implementations paired with their corresponding performance scores (e.g., execution time, throughput, memory efficiency), providing concrete examples of varying solution quality.
- **Generation Protocol:** Explicit instructions defining the required output format and components.
- **Requirements and Restrictions:** Requirements and restrictions to prevent reward hacking in RL.

The model’s response must contain the following three structured components:

- I) **Performance Analysis:** A comparative analysis identifying which previous kernel implementations achieved superior performance scores and the underlying algorithmic or implementation factors responsible for success.
- II) **Algorithm Design:** A high-level description of the proposed optimization strategy, outlining the key techniques to be applied, presented as numbered points in natural language.

III) **Code Implementation:** The complete CUDA kernel implementation incorporating optimizations.

A detailed demonstration for the prompt is shown in Table 3.

2.4.3 Contrastive Exemplar Selection

The selection of code exemplars for prompt construction is critical, as core of Contrastive-RL is to perform meaningful comparative analysis. The selection strategy needs to address the following two key requirements:

1. **Competitive Performance:** The exemplar set should include higher-performing implementations to guide the model toward competitive CUDA codes, avoiding local minima that result from analyzing and comparing inferior codes.
2. **Performance Diversity:** The selected codes must exhibit substantial performance differences to enable effective contrastive analysis.

We employ a sampling strategy akin to that adopted by evolutionary LLM models: Let N denote the number of code exemplars included in each prompt (set to $N = 2$ in our experiments). During RL training, we maintain a performance-indexed database of all successful code samples generated during RL training. Codes are organized into performance buckets B_k based on discretized score intervals, where bucket B_i contains codes with scores in range $[s_k, s_k + \Delta s)$.

We first sample N distinct buckets according to a temperature-scaled softmax distribution:

$$P(B_i) = \frac{\exp((\bar{s}_i - \mu_s)/\tau)}{\sum_j \exp((\bar{s}_j - \mu_s)/\tau)} \quad (1)$$

where $\bar{s}_i$ denotes the aggregate score of bucket B_i , computed as the mean of its constituent code scores, $\mu_s = \text{mean}(\{\bar{s}_j\}_{j=1}^M)$ represents the global mean of all bucket scores, and τ is the temperature parameter governing the exploration-exploitation tradeoff. The sampling strategy in Equation 1 differs from conventional temperature sampling in evolutionary LLM approaches through a modification: the deduction of μ_s stabilizes the distribution by centering scores around zero, which prevents absolute score magnitudes from dominating the selection.

From each selected bucket B_i , we uniformly sample one representative code to construct the final prompt set. This approach satisfies both design criteria: Regarding competitive Performance, score-weighted bucket sampling biases selection toward higher-performing implementations, ensuring the exemplar set contains competitive solutions; Regarding performance Diversity, enforcing selection from N distinct buckets ensures sufficient performance variance for effective contrastive analysis.

A more sophisticated alternative is to use an island-based approach for exemplar selection, as proposed in [21]. However, we find no significant difference in performance between our bucket-based method and the island-based approach. Given this, we opt for the simpler bucket-based strategy.

2.4.4 Reward

In this subsection, we detail the computation of the execution time-based reward function, which serves dual purposes: (1) guiding parameter updates in reinforcement learning and (2) constructing effective prompts. Given a reference CUDA implementation q_i from PyTorch with successful execution time t_{q_i} , and a generated code candidate d with execution time t_d , we define the single-run speedup score as:

$$r_{\text{single-run}}(d) = \frac{t_{q_i}}{t_d} \quad (2)$$

Higher values indicate greater speedup relative to the reference implementation. Evaluations are performed on NVIDIA A100 PCIe. We observe a significant variance in t_d measurements for identical implementations d , which introduces noise in reward estimation. This noise is particularly detrimental to RL training stability. To address these challenges, we implement the following robust measurement strategies:

1. **Dedicated GPU Allocation:** Each evaluation runs on an exclusively allocated GPU. Shared GPU usage leads to significantly higher variance in timing measurements, even when memory and compute utilization appear low.
2. **Paired Execution with Order Randomization:** For fair comparison, each evaluation round executes both the reference q_i and candidate d implementations. Crucially, we randomize the execution order within each round to account for GPU warm-up effects, where subsequent runs typically benefit from cache warming.

CUDA Optimization Task Prompt — Used in Contrastive-RL

Task for CUDA Optimization

You are a CUDA programming expert specializing in GPU kernel optimization. Given a reference CUDA implementation, your objective is to create an accelerated version that maintains identical functionality. You will receive previous CUDA implementations accompanied by their performance metrics. Conduct a comparative analysis of these implementations and use the insights to develop optimized and correct CUDA code that delivers superior performance.

Reference Code

```
1 __global__ void kernel_v1(float* input, float* output, int N) {
2     // Baseline implementation
3     ...
4 }
5 }
```

Previous Cuda Implementations with Scores

```
1 // code1 (score1)
2 __global__ void kernel_v1(float* input, float* output, int N) {
3     ...
4 }
5
6 // code2 (score2)
7 __global__ void kernel_v2(float* input, float* output, int N) {
8     ...
9 }
10 // code3 (score3)
11 __global__ void kernel_v3(float* input, float* output, int N) {
12     ...
13 }
14 // code4 (score4)
15 __global__ void kernel_v3(float* input, float* output, int N) {
16     ...
17 }
```

Generation Protocol

You MUST use exactly two hash symbols (##) at the beginning of each section.

Performance Analysis: Compare code snippets above and articulate on :

1. Which implementations demonstrate superior performance and why?
2. What particular optimization strategies exhibit the greatest potential for improvement?
3. What are the primary performance limitations in the implementation?
4. What CUDA-specific optimization techniques remain unexploited?
5. Where do the most significant acceleration opportunities exist?

Algorithm Design: Describe your optimization approach

Code Implementation: Provide your improved CUDA kernel

Requirements and Restrictions

Critical Requirements:

1. Functionality must match the reference implementation exactly. Failure to do so will result in a score of 0.
2. Code must compile and run properly on modern NVIDIA GPUs

Key Restrictions:

1. Do not cache or reuse previous results — the code must execute fully on each run.
2. Keep hyperparameters unchanged (e.g., batch size, dimensions, etc.) as specified in the reference.

Table 3: Prompt structure for CUDA optimization task showing reference implementations and their performance scores used in Contrastive-RL.

3. **Extended Measurement Window:** We conduct multiple evaluation rounds with predefined running time of 30 minutes per candidate. This adaptive approach yields between several tens of thousands to 1M rounds depending on individual kernel execution times.
4. **Bucketized Variance Control:** We partition all $\text{Score}_{\text{single-run}}(d)$ measurements into 7 buckets and compute bucket-wise averages. Evaluations with inter-bucket variance exceeding 0.005 are discarded.
5. **Robust Central Tendency:** The final reward uses the median of bucket averages, which proves more stable than the mean against outlier effects:

$$r(d) = \text{median}(\{\text{Bucket}_k\}_{k=1}^7) \quad (3)$$

6. **Conservative Rounding:** We apply conservative rounding to speedup ratios (i.e., $\text{Score}(d)$), truncating to two decimal places while biasing toward unity (e.g., $1.118 \rightarrow 1.11$, $0.992 \rightarrow 1.00$).
7. **Strict Verification Protocol:** Despite these precautions, we still occasionally observe spurious large speedups due to GPU turbulence. For any candidate showing either:
 - Absolute value of speedup > 3 , or
 - Speedup exceeding twice the previous maximum
we perform verification on a different GPU of the same type. The result is accepted only if the verification measurement differs by $< 10\%$ from the original.

2.4.5 RL Training

For RL training, we adopt the Group Relative Policy Optimization (GRPO) strategy [23]. Specifically, for each reference prompt q containing selected exemplars as shown in Table 3, we sample G code outputs from the current policy π_{old} , denoted as $\{d_1, d_2, \dots, d_G\}$. Let $\mathbf{r} = (r_1, r_2, \dots, r_G)$ represent the reward scores associated with the generated code samples. Different from standard GRPO training, rewards are smoothed to mitigate the reward hacking issue; the details of this approach will be elaborated in Section 3. Further, as in GRPO, rewards are normalized within each group using:

$$\hat{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})} \quad (4)$$

The complete GRPO objective optimizes the policy model by maximizing:

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E}_{q \sim P(q), \{d_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(d|q)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|d_i|} \sum_{t=1}^{|d_i|} \left(\min \left(\frac{\pi_{\theta}(d_{i,t}|q, d_{i,<t})}{\pi_{\theta_{\text{old}}}(d_{i,t}|q, d_{i,<t})} \hat{r}_i, \right. \right. \right. \\ \left. \left. \left. \text{clip} \left(\frac{\pi_{\theta}(d_{i,t}|q, d_{i,<t})}{\pi_{\theta_{\text{old}}}(d_{i,t}|q, d_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{r}_i \right) - \beta D_{KL}[\pi_{\theta} \parallel \pi_{\text{ref}}] \right) \right] \quad (5)$$

where:

- π_{θ} is the policy model being optimized
- $\pi_{\theta_{\text{old}}}$ is the old policy model from the previous iteration
- ε is the parameter for clipping
- β is the KL penalty coefficient that controls deviation from the reference policy
- D_{KL} denotes the KL divergence between the current and reference policies

We refer readers to [23] for details of GRPO. Model parameters are optimized using the GRPO objective, with contrastive prompts that incorporate comparative examples. This concludes our description of contrastive RL.

3 Mitigating Reward Hacking in RL Training

Reinforcement learning is notorious for exhibiting reward hacking behaviors, where models exploit system vulnerabilities to achieve higher rewards while generating outputs that deviate from the intended objectives. A particularly challenging aspect of these pitfalls is that they cannot be anticipated prior to training and are only discovered during the training process.

3.1 Reward Hacking Cases

During our initial training procedure, we identified the following categories of reward hacking behaviours:

Improper Timing Measurement. KernelBench measures execution time by recording timing events on the main CUDA stream:

```
1 start_event.record(original_model_stream)
2 model(*inputs)
3 end_event.record(original_model_stream)
4 torch.cuda.synchronize(device=device)
```

However, RL-generated code exploits this by creating additional CUDA streams that execute asynchronously. Since KernelBench only monitors the main stream, it fails to capture the actual execution time of operations running on parallel streams. This vulnerability is significant: in our initial implementation, we find that 82 out of 250 (32.8%) RL-generated implementations exploit this timing loophole to appear faster than they actually are, leading to an overall speedup of $18\times$. To address this issue, prompt engineering alone is insufficient. The evaluation methodology should be modified to synchronize all CUDA streams before recording the end time, ensuring accurate performance measurement across all concurrent operations as follows:

```
1 start_event.record(custom_model_stream)
2 custom_model(*inputs)
3 # Wait for all model streams to complete before recording end event
4 if custom_contain_new_streams:
5     for stream in custom_model_streams:
6         custom_model_stream.wait_stream(stream)
7 end_event.record(custom_model_stream)
8 torch.cuda.synchronize(device=device)
```

Lazy Evaluation Another important hacking strategy is lazy evaluation, detected by community users on GitHub: Calling `custom_model(*inputs)` doesn't ensure the output is actually materialized/computed. The computation is actually executed at the correctness check phase when calling the `torch.allclose()` function, allowing it to pass the correctness check.

```
1 class LazyMatmul(torch.Tensor):
2     def __new__(cls, A, B):
3         result = torch.Tensor._make_subclass(cls, torch.empty(0))
4         result.A = A
5         result.B = B
6         result._shape = (A.size(0), B.size(1))
7         return result
8
9     def materialize(self):
10         """Trigger actual computation"""
11         return torch.matmul(self.A, self.B)
12
13 class ModelNew(nn.Module):
14     def forward(self, A, B):
15         return LazyMatmul(A, B) # Returns lazy object
```

To mitigate this issue, we enforce a validation check that ensures materialize functions are called before ending the time measurement, which involves checking the following conditions: the output must be a tensor, must be a standard `torch.Tensor` (not a subclass), must be on the correct device, must have allocated memory, and the corresponding storage must be valid.

```
1 # Check 1: Must be a tensor
2 if not isinstance(out, torch.Tensor):
3     return False, f"{prefix} is not a tensor: {type(out)}"
4
5 # Check 2: Must be standard torch.Tensor, not a subclass
6 if type(out).__name__ not in ['Tensor', 'Parameter']:
7     return False, f"{prefix} is {type(out).__name__}, not standard torch.Tensor"
8
9 # Check 3: Must be on correct device
10 if out.device != device:
11     return False, f"{prefix} on wrong device: {out.device} (expected {device})"
12
13 # Check 4: Must have allocated storage
14 storage_size = out.untyped_storage().size()
15 if storage_size == 0:
16     return False, f"{prefix} has no allocated storage (likely lazy)"
17
18 # Check 5: Storage pointer must be valid
19 ptr = out.data_ptr()
```

```

20 if ptr == 0:
21     return False, f"{prefix} storage pointer is null (likely lazy)"

```

Hyperparameter Manipulation: In KernelBench, each computational task is associated with specific hyperparameters, including `batch_size`, `dim`, `in_features` dimension, `out_features` dimension, `scaling_factor`, and others. The RL agent learned to exploit these parameters by generating code that artificially reduces their values, thereby achieving superficial speedup improvements that do not reflect genuine optimization performance.

Result Caching: The RL agent developed strategies to cache computational results across evaluation batches based on input addresses. When another input’s address matches a cached one, it returns the cached output. In theory, this should not pass correctness validation because the cached output differs from the expected one. However, given that correctness validation checks whether the difference at each position between the reference output and custom code output is below a certain threshold, there are a few cases where it is able to squeeze past the correctness bar. The following code snippet gives an illustration:

```

1 cache_key = x.data_ptr()
2 # Check if result is in cache
3 if cache_key in self.cache:
4     return self.cache[cache_key]

```

3.2 Towards Robust Reward Design and Training Procedures

To mitigate reward hacking, we implement the following strategies during training:

A reward checking model When there is a significant leap in reward, an adversarial model intervenes to determine whether the code exploits the reward system. We use DeepSeek-R1 for this purpose and find that it successfully identifies reward hacking above over 60% of the time.

Hacking-case database We maintain a dynamic hacking-case database that is updated whenever a new reward hacking behavior is detected. The reward checking model leverages this database for detection: given a newly generated code snippet to examine, we retrieve the three most similar cases from the database and include them as context for the reward checking model’s input.

Reward smoothing Sharp reward increases are smoothed to reduce their magnitude, preventing the RL agent from over-prioritizing any single high-reward solution, whether legitimate or not:

$$\begin{aligned}
 r_{\text{normalized}} &= \frac{r - \mu}{\sigma} \\
 r_{\text{smooth}} &= \text{clip}(r_{\text{normalized}}, -k, k)
 \end{aligned} \tag{6}$$

where μ and σ are the mean and the mean and standard deviation of the reward distribution, respectively. k is a hyperparameter that controls the clipping threshold set to 1.5, as we think as achieving a $1.5\times$ speedup over the official PyTorch implementation already represents significant optimization performance.

4 Experiments and Analysis

4.1 KernelBench and Evaluation

Our evaluation is conducted on the KernelBench dataset [20]. The KernelBench Dataset contains a collection of 250 PyTorch workloads designed to evaluate language models’ ability to generate efficient GPU kernels. The dataset is structured across three hierarchical levels based on computational complexity: Level 1 contains 100 tasks with single primitive operations (such as convolutions, matrix multiplications, activations, and normalizations), Level 2 includes 100 tasks with operator sequences that can benefit from fusion optimizations (combining multiple operations like convolution + ReLU + bias), and Level 3 comprises 50 full ML architectures sourced from popular repositories including PyTorch, Hugging Face Transformers, and PyTorch Image Models (featuring models like AlexNet and MiniGPT). Each task in the dataset provides a reference PyTorch implementation with standardized input/output specifications, enabling automated evaluation of both functional correctness and performance through wall-clock timing comparisons. The dataset represents real-world engineering challenges where successful kernel optimization directly translates to practical performance improvements. Throughout this paper, we use KernelBench as the

Configuration	Method	Mean	Max	75%	50%	25%	Success [↑] # out of total	Speedup [↑] >1.01x out of total
<i>Default</i>	All	3.12	120	2.25	1.42	1.17	249/250	226/250
	Level 1	2.78	65.8	1.75	1.28	1.12	99/100	80/100
	Level 2	3.55	120	2.05	1.39	1.20	100/100	98/100
	Level 3	2.96	24.9	2.60	1.94	1.42	50/50	48/50
<i>Torch Compile</i>	All	2.77	69.0	2.55	1.72	1.14	249/250	203/250
	Level 1	3.04	59.7	2.71	1.99	1.41	99/100	89/100
	Level 2	2.91	69.0	1.99	1.55	1.10	100/100	78/100
	Level 3	1.98	8.57	2.28	1.68	1.00	50/50	36/50
<i>Torch Compile RO</i>	All	2.88	80.1	2.48	1.67	1.13	249/250	200/250
	Level 1	3.38	55.3	3.02	2.29	1.61	99/100	90/100
	Level 2	3.00	80.1	2.06	1.54	1.10	100/100	79/100
	Level 3	1.62	8.67	1.76	1.13	0.991	50/50	31/50
<i>CUDA Graph</i>	All	2.81	97.9	1.83	1.20	0.954	249/250	147/229
	Level 1	3.18	59.6	2.09	1.38	1.04	99/100	68/88
	Level 2	2.84	97.9	1.55	1.08	0.932	100/100	53/94
	Level 3	2.06	24.6	1.74	1.08	0.887	50/50	26/47

Table 4: Performance comparison across different configurations on KernelBench on A100. RO = Reduce Overhead. Success and Speedup indicate the number of successful benchmarks out of the total for each level. Note that for CUDA Graph, the total benchmark count differs from the dataset/data-subset size, as some original reference code in KernelBench cannot be successfully transformed into the corresponding CUDA Graph implementations.

evaluation benchmark. KernelBench is recognized as a challenging benchmark in the community [20], with even the best current LLMs improving fewer than 20% of tasks.

For each task with reference implementation q , we evaluate the performance of a generated CUDA code d using a similar protocol to training: We execute both q and d in randomized order within a fixed time budget of 20 minutes per task. The number of execution rounds varies across tasks due to differences in individual runtimes. The final evaluation score for d is computed as the average speedup ratio across all execution rounds within the allocated time window. Unsuccessful implementations receive a score of zero. The metrics we report include speedup statistics (mean, maximum, and 75th, 50th, and 25th percentiles), success rate, and percentage of improvements.

Due to execution time fluctuations, we only consider ratios greater than 1.01 as meaningful speedups.

4.2 Comparison Setups

To perform a comprehensive evaluation on the generated code, we perform the following comparisons:

I) Default This compares the CUDA-L1 generated code with the reference code by KernelBench.

II) Torch Compile This compares the CUDA-L1 generated code with the reference code enhanced by torch.compile with default settings. Torch.compile applies graph-level optimizations including operator fusion, memory planning, and kernel selection to accelerate PyTorch models through just-in-time compilation.

III) Torch Compile Reduce Overhead This compares the CUDA-L1 generated code with the reference code enhanced by torch.compile with reduce-overhead mode enabled. This mode minimizes the compilation overhead by caching compiled graphs more aggressively and reducing recompilation frequency, making it particularly suitable for inference workloads with static shapes.

IV) CUDA Graph Since KernelBench does not provide official CUDA Graph implementations, we employ Claude 4 to generate CUDA Graph-augmented code for each reference implementation. CUDA Graphs capture a series of CUDA kernels and their dependencies into a single graph structure that can be launched with minimal CPU overhead, eliminating the need for

Methods	Model	Mean	Max	75%	50%	25%	Success↑ # out of 250	Speedup↑ >1.01 # out of 250
<i>Vanilla</i>	Llama 3.1-405B	0.23	3.14	0.63	0	0	68	5
	DeepSeek-V3	0.34	2.96	0.76	0	0	99	9
	DeepSeek-R1	0.88	14.4	1.00	0.75	0	179	18
	OpenAI-O1	0.73	12.4	1.00	0.55	0	141	14
<i>Evolve</i>	Llama 3.1-405B	1.18	18.4	1.03	1.00	1.00	247	88
	DeepSeek-V3	1.32	52.4	1.32	1.03	1.00	247	113
	DeepSeek-R1	1.41	44.2	1.45	1.17	1.00	248	162
	OpenAI-O1	1.35	63.9	1.38	1.16	1.00	247	158
<i>CUDA-L1</i>	Stage 1	1.14	32.7	1.00	1.00	0.96	240	50
	Stage 1+2	1.36	48.3	1.41	1.09	1.00	247	175
	Stage 1+2+GRPO	2.41	84.6	1.83	1.33	1.11	247	207
	3 stages - random	2.14	64.5	1.62	1.21	1.09	241	186
	- island	3.21	126	2.21	1.40	1.16	249	223
	- bucket	3.12	120	2.25	1.42	1.17	249	226

Table 5: Model performances on KernelBench All Level.

repeated kernel launch commands and significantly reducing CPU-GPU synchronization costs. Specifically, we provide Claude 4 with the reference code and request the addition of CUDA Graph optimizations. The generated output is then evaluated for correctness. If the code fails validation, we iterate by providing Claude 4 with both the original reference code and the previous erroneous outputs, requesting a corrected version. This iterative process continues for up to 10 attempts until the generated code passes all correctness checks. We release the CUDA Graph codes for KernelBench to the community, providing researchers and practitioners with ready-to-use optimized implementations that can serve as strong baselines for future performance studies and benchmarking efforts.

4.3 Main Results on KernelBench

The experimental results in Table 4 demonstrate CUDA-L1’s optimization effectiveness across different baseline configurations on KernelBench. CUDA-L1 achieves substantial performance improvements over the Default baseline with $3.12\times$ average speedup and $120\times$ maximum gains. Against Torch compilation baselines, CUDA-L1 delivers moderate but consistent improvements with $2.77\text{--}2.88\times$ mean speedup ratios, while demonstrating $2.81\times$ mean improvement over CUDA Graph baseline with notable $97.9\times$ maximum gains.

Across difficulty levels, CUDA-L1’s optimization effectiveness varies by task complexity. For Level 1 (single operations), CUDA-L1 achieves moderate improvements ranging from $2.78\text{--}3.38\times$ over different baselines. Level 2 (operator sequences) shows CUDA-L1’s strongest performance with $3.55\times$ improvement over Default baseline. Level 3 (complex ML tasks) reveals interesting baseline-dependent effectiveness: CUDA-L1 achieves $2.96\times$ improvement over Default baseline, but shows reduced effectiveness against Torch compilation baselines (only $1.62\text{--}1.98\times$ improvements), suggesting these configurations provide stronger baseline performance for complex operations.

Success rates remain consistently high (99.6–100%) across all configurations, while CUDA-L1’s actual speedup achievement rates ($>1.01\times$) demonstrate its optimization reliability: 90.4% success against Default baseline (226/250 cases), 80.0–81.2% against Torch compilation baselines, and 64.2% against CUDA Graph baseline (147/229).

4.4 Baseline Comparison

We compare the results with the following three groups of baselines:

Vanilla Foundation Models: To establish baseline performance benchmarks, we evaluate OpenAI-o1, DeepSeek-R1, DeepSeek-V3, and Llama 3.1-405B Instruct (denoted by **OpenAI-o1-vanilla**, **DeepSeek-R1-vanilla**, **DeepSeek-V3-vanilla** and **Llama 3.1-405B-vanilla**) by prompting each model to optimize the reference CUDA code. The generated CUDA code is directly used for evaluation without further modification. For each task, we repeat this process 5 times and report the best score.

Evolutionary LLM: We implement evolutionary LLM strategies where, given a set of previous codes, we sample up to 4

high-performing kernels based on evaluation scores. The key difference is that the model only performs contrastive analysis without updating model parameters. We adopt the island strategy for code database construction and sampling, as suggested in [18]. We conduct experiments on DeepSeek-R1, OpenAI-o1 and Llama 3.1-405B, denoted as **DeepSeek-R1-evolve**, **OpenAI-o1-evolve**, **DeepSeek-V3-evolve** and **Llama 3.1-405B-evolve**.

Different combinations of CUDA-L1 components and variants:

- **stage1**: Uses only the outcome from the first stage with supervised fine-tuning applied
- **stage1+2**: Applies only the first two stages without reinforcement learning
- **stage1+2 + GRPO**: Replaces the contrastive RL with a vanilla GRPO strategy, without comparative analysis
- **random sampling**: Replaces the bucket sampling strategy with simple random sampling of exemplars
- **island sampling**: Adopts an island-based sampling strategy [18], where examples are distributed across different islands, prompts are constructed using exemplars from the same island, and newly generated examples are added to that island. After a fixed number of iterations, examples in half of the inferior islands are eliminated and examples from superior islands are copied to replace them.

Results are shown in Table 5. As observed, all vanilla foundation models perform poorly on this task. Even the top-performing models—DeepSeek-R1 and OpenAI-o1—achieve speedups over the reference kernels in fewer than 10% of tasks, while Llama 3.1-405B optimizes only 2.4% of tasks. This confirms that vanilla foundation models cannot be readily applied to CUDA optimization due to their insufficient grasp of CUDA programming principles and optimization techniques.

We observe significant performance improvements introduced by the Evolutionary LLM models compared to vanilla foundation model setups, despite sharing the same parameter sets. All Evolve models achieve speedups in over 70% of tasks, with DeepSeek-R1 reaching 72.4% success rate. This demonstrates that leveraging contrastive analysis, which exploits the model’s general reasoning abilities, is more effective than direct output generation. The superiority of evolutionary LLM over vanilla LLM also provides evidence that contrastive RL should outperform non-contrastive RL approaches like vanilla GRPO, as the relationship between evolutionary and vanilla LLMs parallels that between contrastive and non-contrastive RL methods.

When comparing the different combinations of CUDA-L1 components, we observe a progressive increase in speedup rates from **stage1 (SFT only)** at 22.4% to **stage1+2 (SFT + self-supervised)** at 66%, and further to **stage1+2+GRPO** at 88.4%. This demonstrates the cumulative benefits of each training stage in improving model performance.

When comparing different database construction and exemplar sampling strategies, both the bucket-sampling strategy (96% speedup rate) and the island-based strategy (95.2% speedup rate) achieve near-optimal performance, with both significantly outperforming the random sampling strategy (82.4% speedup rate). This aligns with our expectations, as competitive exemplars must be included in the prompt to guide the model toward generating more competitive solutions.

All RL-based approaches significantly outperform evolutionary LLM baselines with fixed model parameters, with the best RL methods achieving over 95% speedup rates compared to 72.4% for the best evolutionary approach. This demonstrates the necessity of model parameter updating for achieving optimal performance in CUDA optimization tasks. While evolutionary approaches can leverage reasoning capabilities through contrastive analysis, fine-tuning the model parameters allows for deeper adaptation to the specific domain knowledge and optimization patterns required for effective CUDA kernel generation. The performance gap suggests that domain-specific parameter adaptation is crucial for bridging the gap between general reasoning abilities and specialized code optimization expertise.

4.5 Generalization of A100-Optimized Kernels to Other GPU Architectures

Even without being specifically tailored to other GPU architectures, we observe significant performance improvements across all tested GPU types, with mean speedups ranging from $2.38\times$ to $3.85\times$. H100 achieves the highest mean speedup ($3.85\times$) with exceptional maximum gains ($368\times$), while A100 PCIe and L40 demonstrate strong performance with mean speedups of $3.12\times$ and $3.13\times$ respectively. L40 shows the second-highest maximum speedup ($182\times$) among all GPUs. The consumer RTX 3090 achieves a competitive mean speedup of $2.51\times$, while H20 shows moderate performance with $2.38\times$ mean speedup. Notably, A100 maintains the highest 75th percentile ($2.25\times$), 50th percentile ($1.42\times$), and 25th percentile ($1.17\times$) values, indicating more consistent optimization performance on the target architecture.

The success rates remain high across all architectures (242-250 out of 250), with H100 achieving perfect success (250/250), validating that CUDA optimization techniques can generalize across different GPU architectures. Speedup achievement rates ($>1.01\times$) vary by architecture, with H20 and A100 showing the highest effectiveness (226 and 226 successful optimizations respectively), while RTX 3090 demonstrates good performance with 201 successful optimizations.

Configuration	GPU Device	Mean	Max	75%	50%	25%	Success ↑ # out of 250	Speedup ↑ >1.01x
<i>Default</i>	A100	3.12	120	2.25	1.42	1.17	249	226/250
	3090	2.51	114	1.57	1.18	1.03	242	201/250
	H100	3.85	368	1.76	1.32	1.09	250	218/250
	H20	2.38	63.7	1.81	1.34	1.11	247	226/250
	L40	3.13	182	1.88	1.31	1.08	248	215/250
<i>Torch Compile</i>	A100	2.77	69.0	2.55	1.72	1.14	249	203/250
	3090	2.58	73.2	2.23	1.50	1.00	242	177/250
	H100	2.74	49.7	2.83	1.92	1.11	250	195/250
	H20	2.89	49.4	3.21	2.04	1.19	247	209/250
	L40	2.85	96.9	2.43	1.82	1.13	248	199/250
<i>Torch Compile RO</i>	A100	2.88	80.1	2.48	1.67	1.13	249	200/250
	3090	2.61	72.9	2.29	1.48	1.00	242	172/250
	H100	2.77	61.2	2.78	1.61	1.00	247	187/250
	H20	2.82	52.1	3.18	1.64	1.06	247	192/250
	L40	2.89	90.9	2.54	1.72	1.08	248	193/250
<i>CUDA Graph</i>	A100	2.81	97.9	1.83	1.20	0.954	229	147/229
	3090	3.34	156	1.94	1.28	0.997	206	148/206
	H100	2.23	70.1	1.60	1.04	0.838	222	119/222
	H20	2.20	64.6	1.69	1.09	0.854	229	133/229
	L40	3.98	275	1.83	1.16	0.862	224	137/224

Table 6: Performance comparison across different configurations and GPU devices on KernelBench. RO = Reduce Overhead. Speedup is defined as value exceeding 1.01x.

Level ID	Task ID	Task Name	Speedup
2	83	83_Conv3d_GroupNorm_Min_Clamp_Dropout	120.3
1	12	12_Matmul_with_diagonal_matrices	64.4
2	80	80_Gemm_Max_Subtract_GELU	31.3
1	9	9_Tall_skinny_matrix_multiplication	24.9
3	31	31_VisionAttention	24.8
2	96	96_ConvTranspose3d_Multiply_Max_GlobalAvgPool_Clamp	16.2
2	66	66_Matmul_Dropout_Mean_Softmax	14.5
1	13	13_Matmul_for_symmetric_matrices	14.4
3	43	43_MinGPTCausalAttention	13.1
3	44	44_MiniGPTBlock	10.5

Table 7: KernelBench Tasks Ranked by RL-CUDA1 Acceleration (Top 10)

These results demonstrate that while A100-optimized kernels transfer to other GPUs with varying degrees of effectiveness, the optimizations achieve substantial improvements across architectures. H100’s exceptional performance suggests strong compatibility with the optimization techniques, while A100’s consistent percentile performance validates the target architecture optimization. The varying maximum speedups ($63.7\times$ to $368\times$) across GPUs indicate architecture-specific optimization potential, suggesting that dedicated optimizations for each GPU type would further enhance performance. We plan to release kernels specifically trained for different GPU types in an updated version of CUDA-L1.

4.6 Discovered Cuda Optimization Techniques

An analysis of optimization strategies commonly employed in enhanced CUDA implementations reveals interesting patterns. Through GPT-4o-based technical term extraction and frequency analysis, we identified the ten most prevalent optimization techniques:

- **Memory Layout Optimization**, which ensures data is stored in contiguous memory blocks;
- **Memory Access Optimization**, which arranges data access patterns to maximize memory bandwidth and minimize latency through techniques like shared memory usage, coalesced global memory access, and memory padding;
- **Operation Fusion**, which combines multiple sequential operations into a single optimized kernel execution;
- **Memory Format Optimization**, which aligns data layout with hardware memory access patterns;
- **Memory Coalescing**, which optimizes CUDA kernel performance by ensuring threads in the same warp access contiguous memory locations;
- **Warp-Level Optimization**, which leverages the parallel execution of threads within a warp (typically 32 threads) to efficiently perform collective operations;
- **Optimized Thread Block Configuration**, which carefully selects grid and block dimensions for CUDA kernels to maximize parallel execution efficiency and memory access patterns;
- **Shared Memory Usage**, enables fast data access by storing frequently used data in a cache accessible by all threads within a thread block;
- **Register Optimization**, which stores frequently accessed data in fast register memory to reduce latency and improve computational throughput;
- **Stream Management**, which enables parallel execution of operations for improved GPU utilization.

Tables 11, 13 and 14 present detailed CUDA optimization techniques with accompanying code examples.

5 Case Studies

Table 7 presents the KernelBench tasks that achieved the highest speedups. We examine these some of them in detail and perform an ablation study of the applied CUDA optimization techniques, showing how much each technique contributes to the final speedup.

Configuration	CUDA Graphs	Memory Contiguity	Static Tensor Reuse	Speedup	Bottleneck
CUDA + Memory + Static	✓	✓	✓	3.42×	LSTM computation
CUDA + Memory	✓	✓	✗	2.96×	Memory allocation
CUDA + Static	✓	✗	✓	2.84×	Memory layout
CUDA Only	✓	✗	✗	2.77×	Memory overhead
Memory + Static	✗	✓	✓	1.00×	Kernel launch overhead
Memory Only	✗	✓	✗	1.00×	Kernel launch overhead
Static Only	✗	✗	✓	1.00×	Kernel launch overhead
Baseline	✗	✗	✗	1.00×	Kernel launch overhead

Table 8: Speedup achieved by different CUDA optimization techniques on LSTMs.

5.1 $\text{diag}(A) * B$: 64× faster

We first examine the code for level 1, task 12, which performs matrix multiplication between a diagonal matrix (represented by its diagonal elements) and a dense matrix, both with dimension $N=4096$. The reference code is as follows where `__init__` function of the class is omitted:

```

1 class Model(nn.Module):
2     def forward(self, A, B):
3         # A: (N,) - 1D tensor of shape N
4         # B: (N, M) - 2D tensor of shape N x M
5         # torch.diag(A): (N, N) - creates diagonal matrix from A
6         # Result: (N, N) @ (N, M) = (N, M)
7         return torch.diag(A) @ B

```

Let’s see the optimized code by CUDA-11:

```

1 class Model(nn.Module):
2     def forward(self, A, B):
3         return A.unsqueeze(1) * B

```

The optimized implementation leverages PyTorch’s broadcasting mechanism to perform diagonal matrix multiplication efficiently. It first reshapes the diagonal vector A from shape $(N,)$ to $(N, 1)$ using `unsqueeze(1)`, transforming it into a column vector. Next, it utilizes PyTorch’s automatic broadcasting to multiply each row of matrix B by the corresponding element of A , where the $(N, 1)$ shaped tensor is implicitly expanded to match the (N, M) dimensions of B . This approach completely avoids creating the full $N \times N$ diagonal matrix, which would be sparse and memory-intensive. The key benefits of this technique are substantial: it requires only $O(1)$ extra memory instead of $O(N^2)$ for storing the diagonal matrix, reduces computational complexity from $O(N^2M)$ operations for full matrix multiplication to just $O(NM)$ element-wise operations, leading to 64× speedup.

What makes this particularly valuable is that RL can systematically explore the vast space of equivalent implementations. By exploring semantically equivalent implementations, RL learns to identify patterns where computationally expensive operations can be replaced with more efficient alternatives. The power of RL extends beyond simple algebraic simplifications and it can uncover sophisticated optimizations such as: replacing nested loops with vectorized operations identifying hidden parallelization opportunities discovering memory-efficient mathematical reformulations finding non-obvious algorithmic transformations that preserve correctness while improving performance What makes this particularly valuable is that RL can systematically explore the vast space of equivalent implementations—something that would be impractical for human engineers to do manually.

5.2 LSMT: 3.4× faster

Now let’s look at a classical neural network algorithm LSTM (level 3, task 35), on which CUDA-11 achieves a speedup of 3.4×. By comparing the reference PyTorch implementation with the optimized output, we identified the following optimization techniques:

1. **CUDA Graphs**, which captures the entire LSTM computation sequence (including all layer operations) into a replayable graph structure, eliminating kernel launch overhead by recording operations once and replaying them with minimal CPU involvement for subsequent executions.
2. **Memory Contiguity**, which ensures all tensors maintain contiguous memory layouts through explicit `.contiguous()` calls before operations, optimizing memory access patterns and improving cache utilization for CUDA kernels processing sequential data.
3. **Static Tensor Reuse**, which pre-allocates input and output tensors during graph initialization and reuses them across forward passes with non-blocking copy operations, eliminating memory allocation overhead and enabling asynchronous data transfers.

Table 8 represents the results for 8 different optimization combinations across the three optimization techniques above. As can be seen, CUDA Graphs is essential for achieving any meaningful speedup in this LSTM model. All configurations with CUDA Graphs achieve 2.77x-3.42x speedup, while all configurations without it achieve only 1.0x (no speedup). The combination of all three techniques provides the best performance at 3.42x, demonstrating that while CUDA Graphs provides the majority of the benefit (81% of total speedup), the additional optimizations contribute meaningful improvements when combined together.

5.3 3D transposed convolution: 120× faster

We examined the code for Level 2, Task 38, which implements a sequence of 3D operations: transposed convolution, average pooling, clamping, softmax, and element-wise multiplication. By comparing the reference PyTorch implementation with the CUDA-L1 optimized output, we identified the following optimization techniques applied by CUDA-L1:

1. **Mathematical Short-Circuit**, which detects when `min_value` equals 0.0 and skips the entire computation pipeline (convolution, normalization, min/clamp operations), directly returning zero tensors since the mathematical result is predetermined.
2. **Pre-allocated Tensors**, which creates zero tensors of standard shapes during initialization and registers them as buffers, eliminating memory allocation overhead during forward passes for common input dimensions.
3. **Direct Shape Matching**, which provides a fast path for standard input shapes by immediately returning pre-allocated tensors without any shape calculations, bypassing the computational overhead entirely.
4. **Pre-computed Parameters**, which extracts and stores convolution parameters (kernel size, stride, padding, dilation) during initialization, avoiding repeated attribute lookups and tuple conversions during runtime.

Table 9 represents the results for 16 different optimization combinations across the four optimization techniques above. As can be seen, mathematical short-circuit is essential for this task, where all configurations with mathematical short-circuit achieve 28.6x+ speedup, while all configurations without it achieve only 1.0x (no).

The fact that CUDA-L1 identified this precise optimization strategy demonstrates the power of reinforcement learning in navigating complex optimization spaces. While a human developer might intuitively focus on computational optimizations (like parallel algorithms) or memory layout improvements (like tensor pre-allocation), RL discovered that the mathematical properties of the operation completely dominate performance. This discovery is particularly impressive because: RL is able to find this non-obvious solution: The 120x speedup from exploiting the mathematical short-circuit is counterintuitive as most developers would expect to optimize the convolution kernel or memory access patterns for such a compute-heavy operation. This shows how RL can discover optimal solutions that challenge conventional wisdom in deep learning optimization. Where human intuition might suggest "optimize the convolution algorithm first," CUDA-L1 learned through empirical evidence that "recognize when computation can be entirely skipped" yields dramatically better results. The agent's ability to identify that `min(x, 0)` followed by `clamp(0, 1)` always produces zeros demonstrates how RL can uncover mathematical invariants that humans might overlook in complex computational pipelines.

6 Related Work

6.1 RL-augmented LLMs for Code Optimization

Starting this year, there has been a growing interest in using LLM or RL-augmented LLM models for code optimization, including recent work on compiler optimization [5] and assembly code optimization [28], which use speed and correctness as RL training rewards. Other more distant related is software optimization that scale RL-based LLM reasoning for software engineering [30]. Regarding CUDA optimization, the only work that comprehensively delves into KernelBench is from [13], which uses a meta-generation procedure that successfully optimizes 186 tasks out of 250 tasks in KernelBench with a medium speedup of 34%. Other works remain in preliminary stages, including [4], which has optimized 20 GPU kernels selected from

Configuration	Math Short-Circuit	Pre-allocated Tensors	Direct Shape Match	Pre-computed Params	Speedup
Math + PreAlloc + Shape + Params	✓	✓	✓	✓	120.9×
Math + Shape + Params	✓	✗	✓	✓	32.8×
Math + PreAlloc + Params	✓	✓	✗	✓	30.6×
Math + Params	✓	✗	✗	✓	30.2×
Math + PreAlloc	✓	✓	✗	✗	29.2×
Math Only	✓	✗	✗	✗	29.2×
Math + Shape	✓	✗	✓	✗	28.6×
Math + PreAlloc + Shape	✓	✓	✓	✗	28.6×
PreAlloc + Shape + Params	✗	✓	✓	✓	1.0×
PreAlloc + Shape	✗	✓	✓	✗	1.0×
Shape + Params	✗	✗	✓	✓	1.0×
PreAlloc + Params	✗	✓	✗	✓	1.0×
Params Only	✗	✗	✗	✓	1.0×
Shape Only	✗	✗	✓	✗	1.0×
PreAlloc Only	✗	✓	✗	✗	1.0×
Baseline	✗	✗	✗	✗	1.0×

Table 9: Speedup achieved by different CUDA optimization techniques on the Conv3d task.

three different sources: the official NVIDIA CUDA Samples, LeetGPU, and KernelBench using a proposed feature search and reinforcement strategy; and an ongoing tech report [22] that optimizes 4 kernels.

6.2 Evolutionary LLMs

Evolutionary large language models [33, 16, 21, 18, 28, 6, 14] represent a paradigm shift in automated algorithm discovery, exemplified by systems such as Google DeepMind’s AlphaEvolve [18] and FunSearch [21]. These systems harness LLMs and operate through an iterative evolutionary process:

1. **A program sampler** samples the high-score programs from previous generations to construct the prompt. Programs are usually sampled based on scores to promote diversity.
2. **An LLM** that generates new algorithmic variants based on the generated prompt.
3. **An automated evaluator** tests and scores the generated programs for correctness and performance.
4. **An evolutionary database** stores successful candidates and selects the most promising ones for future iterations. In a more sophisticated setup called island-based evolution, candidates from low-performing islands are wiped out from the database after a finite number of iterations, while candidates from high-performing islands are migrated to repopulate the wiped islands.

Evolutionary LLMs operates iteratively and progressively improves algorithm performance This methodology has achieved breakthroughs including new matrix multiplication algorithms surpassing Strassen’s 1969 approach and practical optimizations for Google’s data centers, demonstrating the system’s ability to evolve.

7 Conclusion

In this paper, we propose CUDA-L1, a pipelined system for CUDA optimization powered by contrastive RL. CUDA-L1 achieves significant performance improvements on the CUDA optimization task, delivering an average speedup of $\times 3.12$ (median $\times 1.42$) over the default baseline across all 250 CUDA kernels of KernelBench, with peak speedups reaching $\times 120$ on A100. Against other baselines, CUDA-L1 demonstrates $\times 2.77$ over Torch Compile, $\times 2.88$ over Torch Compile with reduce overhead, and $\times 2.81$ over CUDA Graph implementations. CUDA-L1 can independently discover CUDA optimization techniques, learn to combine them strategically, and more importantly, extend the acquired CUDA reasoning abilities to unseen kernels with meaningful speedups. We hope that CUDA-L1 would open new doors for automated optimization of CUDA, and

substantially promote GPU efficiency and alleviate the rising pressure on GPU computing resources.

A Case Study: Code Snippets Before and After Optimizations

Tech + Desc	Before optimization	After optimization
Memory Layout Optimization Memory Layout Optimization ensures data is stored in contiguous memory blocks to maximize cache efficiency and reduce memory access latency during GPU computations.	- Non-contiguous memory access <pre> 1 '''Python 2 def matrix_multiply(A, B): 3 # A and B might not be contiguous in memory 4 C = torch.mm(A, B) 5 return C 6 ''' </pre>	- Ensuring contiguous memory layout <pre> 1 '''Python 2 def matrix_multiply_optimized(A, B): 3 # Ensure contiguous memory layout for efficient 4 # access patterns 5 A = A.contiguous() if not A.is_contiguous() else A 6 B = B.contiguous() if not B.is_contiguous() else B 7 C = torch.mm(A, B) 8 return C 9 ''' </pre>
Memory Coalescing Memory coalescing optimizes GPU memory access by ensuring threads in a warp access contiguous memory locations, reducing memory transactions and increasing bandwidth utilization.	- Uncoalesced memory access <pre> 1 '''cuda 2 __global__ void uncoalesced_kernel(float* input, 3 float* output) { 4 int tid = threadIdx.x; 5 int stride = blockDim.x; 6 // Each thread accesses non-contiguous memory 7 // locations 8 for (int i = 0; i < 1024; i++) { 9 output[tid + i * stride] = input[tid + i * 10 stride] * 2.0f; 11 } 12 } 13 ''' </pre>	- Coalesced memory access with loop unrolling <pre> 1 '''cuda 2 __global__ void coalesced_kernel(float* input, 3 float* output) { 4 int tid = threadIdx.x; 5 int batch_idx = blockDim.x; 6 // Base pointers for this batch item 7 const float* batch_input = input + batch_idx * 8 1024; 9 float* batch_output = output + batch_idx * 1024; 10 // Each thread processes contiguous memory in 11 // chunks 12 #pragma unroll 4 13 for (int i = 0; i < 1024; i += 16) { 14 batch_output[i] = batch_input[i] * 2.0f; 15 batch_output[i+1] = batch_input[i+1] * 2.0f; 16 batch_output[i+2] = batch_input[i+2] * 2.0f; 17 // ... more contiguous accesses 18 batch_output[i+15] = batch_input[i+15] * 2.0f; 19 } 20 } 21 ''' </pre>
Warp-Level Optimizations Warp-Level Optimizations leverage the CUDA execution model where threads execute in groups of 32 (warps) to improve parallel efficiency through collaborative operations and memory access patterns.	- Each thread independently calculates min value <pre> 1 '''cuda 2 __global__ void min_kernel_before(const float* 3 input, float* output, int size) { 4 int idx = blockIdx.x * blockDim.x + threadIdx.x; 5 if (idx < size) { 6 float min_val = 1e10f; 7 for (int i = 0; i < DEPTH; i++) { 8 min_val = min(min_val, input[idx + i * 9 size]); 10 } 11 output[idx] = min_val; 12 } 13 } 14 ''' </pre>	- Using warp-level operations for parallel reduction <pre> 1 '''cuda 2 3 __global__ void min_kernel_after(const float* 4 input, float* output, int size) { 5 int idx = blockIdx.x * blockDim.x + threadIdx.x; 6 int lane_id = threadIdx.x % 32; // Thread's 7 // position within warp 8 int warp_id = threadIdx.x / 32; // Warp number 9 // within the block 10 float min_val = 1e10f; 11 if (idx < size) { 12 // Each thread finds its local minimum 13 for (int i = 0; i < DEPTH; i++) { 14 min_val = min(min_val, input[idx + i * 15 size]); 16 } 17 // Warp-level parallel reduction using shuffle 18 for (int offset = 16; offset > 0; offset /= 19 2) { 20 float other = __shfl_down_sync(0xffffffff, 21 min_val, offset); 22 min_val = min(min_val, other); 23 } 24 // First thread in warp writes the result 25 if (lane_id == 0) { 26 output[blockIdx.x * (blockDim.x/32) + 27 warp_id] = min_val; 28 } 29 } 30 } 31 ''' </pre>

Table 10: (Part 1) Code snippets before and after optimizations.

Tech + Desc	Before optimization	After optimization
Memory Hierarchy Optimization Memory Hierarchy Optimization involves strategically utilizing different levels of GPU memory (registers, shared memory, constant memory) to minimize global memory access latency and maximize data reuse.	- Using global memory directly <pre> 1 ````cuda 2 __global__ void depthwise_separable_conv_kernel_unoptimized(3 const float* input, const float* depthwise_weight, const float* pointwise_weight, 4 float* output, /* other parameters */) { 5 6 int out_y = blockIdx.y * blockDim.y + threadIdx.y; 7 int out_x = blockIdx.x * blockDim.x + threadIdx.x; 8 9 // Each thread directly accesses global memory for each computation 10 for (int oc = 0; oc < out_channels; oc++) { 11 float result = 0.0f; 12 for (int ic = 0; ic < in_channels; ic++) { 13 float depthwise_result = 0.0f; 14 // Direct global memory access for each kernel element 15 for (int ky = 0; ky < 3; ky++) { 16 for (int kx = 0; kx < 3; kx++) { 17 int in_y = out_y * stride + ky - padding; 18 int in_x = out_x * stride + kx - padding; 19 if (in_y >= 0 && in_y < in_height && in_x >= 0 && in_x < in_width) { 20 depthwise_result += input[(batch_idx * out_channels + oc) * in_channels + ic) * in_height + in_y) * in_width + in_x] * 21 depthwise_weight[ic * 9 + ky * 3 + kx]; 22 } 23 } 24 } 25 result += depthwise_result * pointwise_weight[oc * in_channels + ic]; 26 } 27 output[(batch_idx * out_channels + oc) * out_height + out_y) * out_width + out_x] = result; 28 } 29 } 30 ```` </pre>	- Using memory hierarchy (shared, constant, registers) <pre> 1 ````cuda 2 __constant__ float c_depthwise_weight[3*3*3]; // Constant memory for weights 3 __constant__ float c_pointwise_weight[3*64]; 4 5 __global__ void depthwise_separable_conv_kernel_optimized(6 const float* input, float* output, /* other parameters */) { 7 8 // Shared memory for input tile with padding 9 __shared__ float shared_input[3][SHARED_MEM_HEIGHT][SHARED_MEM_STRIDE]; 10 11 // Collaborative loading of input data to shared memory 12 // [shared memory loading code...] 13 __syncthreads(); 14 15 // Register caching for intermediate results 16 float depthwise_results[3]; // Store in registers 17 18 // Compute using shared memory and constant memory 19 for (int c = 0; c < in_channels; ++c) { 20 float sum = 0.0f; 21 // Fully unrolled convolution using shared memory 22 sum += shared_input[c][sm_y_base][sm_x_base] * c_depthwise_weight[c*9 + 0]; 23 // [more unrolled operations...] 24 depthwise_results[c] = sum; // Store in register 25 } 26 27 // Cache output values in registers 28 float output_cache[32]; 29 30 // Compute pointwise convolution using registers and constant memory 31 for (int i = 0; i < oc_limit; ++i) { 32 output_cache[i] = depthwise_results[0] * c_pointwise_weight[i * 3 + 0] + 33 depthwise_results[1] * c_pointwise_weight[i * 3 + 1] + 34 depthwise_results[2] * c_pointwise_weight[i * 3 + 2]; 35 } 36 37 // Coalesced write to global memory 38 for (int i = 0; i < oc_limit; ++i) { 39 output[output_idx] = output_cache[i]; 40 } 41 } 42 ```` </pre>
Asynchronous Execution Asynchronous Execution in CUDA allows operations to be queued and executed concurrently on separate streams, enabling overlapping computation with memory transfers for improved GPU utilization.	- Sequential execution <pre> 1 ````Python 2 def forward(self, x): 3 # Operations execute in the default stream, blocking sequentially 4 result = self.conv_transpose3d(x) 5 return result 6 ```` </pre>	- Asynchronous execution with custom stream <pre> 1 ````Python 2 def forward(self, x): 3 # Create dedicated compute stream 4 self.compute_stream = torch.cuda.Stream(priority=-1) # High priority stream 5 6 # Execute operations asynchronously in the custom stream 7 with torch.cuda.stream(self.compute_stream): 8 result = self.optimized_cuda_forward(x, x.dtype) 9 10 # Control returns immediately while computation continues in background 11 return result 12 ```` </pre>

Table 11: (Part 2) Code snippets before and after optimizations.

Tech + Desc	Before optimization	After optimization
Memory Access Optimization Memory Access Optimization in CUDA improves performance by organizing data access patterns to maximize cache utilization and minimize memory latency through techniques like tiling, coalescing, and shared memory usage.	- Naive matrix multiplication with poor memory access <pre> 1 ````cuda 2 // Before optimization - Naive matrix multiplication with poor memory access 3 __global__ void matmul_naive(float* A, float* B, float* C, int M, int N, int K) { 4 int row = blockIdx.y * blockDim.y + threadIdx.y; 5 int col = blockIdx.x * blockDim.x + threadIdx.x; 6 7 if (row < M && col < N) { 8 float sum = 0.0f; 9 for (int k = 0; k < K; ++k) { 10 sum += A[row * K + k] * B[col * K + k]; 11 } 12 C[row * N + col] = sum; 13 } 14 } 15 ```` </pre>	- Using shared memory tiling and register blocking <pre> 1 ````cuda 2 __global__ void matmul_optimized(float* A, float* B, float* C, int M, int N, int K) { 3 // Block index and thread index 4 const int bx = blockIdx.x; 5 const int by = blockIdx.y; 6 const int tx = threadIdx.x; 7 const int ty = threadIdx.y; 8 9 // Output positions 10 const int row = by * 8 + ty; 11 const int col = bx * 32 + tx; 12 13 // Register accumulation 14 float sum00 = 0.0f, sum01 = 0.0f; 15 float sum10 = 0.0f, sum11 = 0.0f; 16 17 // Shared memory tiles with padding to avoid bank conflicts 18 __shared__ float As[8][33]; 19 __shared__ float Bs[32][33]; 20 21 // Loop over tiles 22 for (int tile = 0; tile < (K + 31) / 32; ++tile) 23 { 24 // Collaborative loading of tiles into shared memory 25 if (row < M && tile * 32 + tx < K) 26 As[ty][tx] = A[row * K + tile * 32 + tx]; 27 else 28 As[ty][tx] = 0.0f; 29 30 if (col < N && tile * 32 + ty < K) 31 Bs[ty][tx] = B[col * K + tile * 32 + ty]; 32 else 33 Bs[ty][tx] = 0.0f; 34 35 __syncthreads(); 36 37 // Compute partial dot products using shared memory 38 #pragma unroll 8 39 for (int k = 0; k < 32; ++k) { 40 float a0 = As[ty][k]; 41 float a1 = As[ty + 4][k]; 42 float b0 = Bs[k][tx]; 43 float b1 = Bs[k][tx + 16]; 44 45 sum00 += a0 * b0; 46 sum01 += a0 * b1; 47 sum10 += a1 * b0; 48 sum11 += a1 * b1; 49 } 50 __syncthreads(); 51 } 52 53 // Write results to global memory 54 if (row < M && col < N) C[row * N + col] = sum00; 55 if (row < M && col + 16 < N) C[row * N + col + 16] = sum01; 56 if (row + 4 < M && col < N) C[(row + 4) * N + col] = sum10; 57 if (row + 4 < M && col + 16 < N) C[(row + 4) * N + col + 16] = sum11; 58 } 59 ```` </pre>
Operation Fusion Operation Fusion combines multiple consecutive operations into a single optimized kernel to reduce memory transfers and improve computational efficiency on CUDA devices.	- Separate operations <pre> 1 ````Python 2 def forward(self, x): 3 x = F.max_pool3d(x, kernel_size=self.pool_kernel_size, stride=self.pool_stride) 4 x = torch.softmax(x, dim=1) 5 x = x - self.subtract.view(1, -1, 1, 1, 1) 6 x = x * torch.sigmoid(x) 7 return torch.max(x, dim=1)[0] 8 ```` </pre>	- Fused operations with JIT <pre> 1 ````Python 2 @torch.jit.script 3 def fused_post_process(x, subtract_view): 4 x = torch.softmax(x, dim=1) 5 x = x - subtract_view 6 x = x * torch.sigmoid(x) 7 return torch.max(x, dim=1)[0] 8 9 def forward(self, x): 10 x = F.max_pool3d(x, kernel_size=self.pool_kernel_size, stride=self.pool_stride) 11 return self.fused_post_process(x, self.subtract.view(1, -1, 1, 1, 1)) 12 ```` </pre>

Table 12: (Part 3) Code snippets before and after optimizations.

Tech + Desc	Before optimization	After optimization
Optimized Thread Block Configuration Optimized Thread Block Configuration involves carefully selecting grid and block dimensions for CUDA kernels to maximize parallelism, memory access efficiency, and computational throughput based on the hardware architecture and algorithm characteristics.	- Basic thread block configuration <pre> 1 '''Python 2 block_dim = (16, 16) # Simple square thread block 3 grid_dim = (math.ceil(N / 16), math.ceil(M / 16)) 4 5 kernel(grid=grid_dim, block=block_dim, 6 args=[A.data_ptr(), B.data_ptr(), C.data_ptr(), 7 M, N, K]) 8 ''' </pre>	- Carefully tuned thread block configuration <pre> 1 '''Python 2 block_dim = (32, 8) # Rectangular block optimized 3 for matrix multiplication 4 grid_dim = (math.ceil(N / 32), math.ceil(M / 8)) 5 6 kernel(grid=grid_dim, block=block_dim, 7 args=[A.data_ptr(), B.data_ptr(), C.data_ptr(), 8 M, N, K]) 9 ''' </pre>
Branchless Implementation Branchless implementation replaces conditional statements with mathematical operations to avoid branch divergence and improve GPU performance.	- With branches <pre> 1 '''cuda 2 if (val > 1.0f) { 3 output = 1.0f; 4 } else if (val < -1.0f) { 5 output = -1.0f; 6 } else { 7 output = val; 8 } 9 ''' </pre>	- Branchless <pre> 1 '''cuda 2 output = fmaxf(-1.0f, fminf(1.0f, val)); 3 ''' </pre>
Shared Memory Usage Shared memory in CUDA allows threads within the same block to efficiently share data, reducing global memory accesses and improving performance for algorithms with data reuse patterns.	- Each thread reads diagonal element from global memory <pre> 1 '''cuda 2 __global__ void 3 diag_matmul_kernel_unoptimized(const float* A, 4 const float* B, float* C, int N, int M) { 5 int row = blockIdx.y * blockDim.y + threadIdx.y; 6 int col = blockIdx.x * blockDim.x + threadIdx.x; 7 8 if (row < N && col < M) { 9 // Each thread loads the same diagonal 10 // element multiple times from global memory 11 C[row * M + col] = A[row] * B[row * M + col]; 12 } 13 } 14 ''' </pre>	- Using shared memory to cache diagonal elements <pre> 1 '''cuda 2 __global__ void diag_matmul_kernel_optimized(const 3 float* A, const float* B, float* C, int N, int 4 M) { 5 const int BLOCK_SIZE_Y = 8; 6 __shared__ float A_shared[BLOCK_SIZE_Y]; // 7 Shared memory for diagonal elements 8 9 int row = blockIdx.y * blockDim.y + threadIdx.y; 10 int col = blockIdx.x * blockDim.x + threadIdx.x; 11 12 // Load diagonal elements into shared memory 13 // (once per row in block) 14 if (threadIdx.x == 0 && row < N) { 15 A_shared[threadIdx.y] = A[row]; 16 } 17 18 __syncthreads(); // Ensure all threads see the 19 loaded values 20 21 if (row < N && col < M) { 22 // Use cached diagonal element from shared 23 // memory 24 C[row * M + col] = A_shared[threadIdx.y] * 25 B[row * M + col]; 26 } 27 } 28 ''' </pre>
Minimal Synchronization Minimal Synchronization reduces overhead by minimizing the number of synchronization points between CPU and GPU operations, allowing asynchronous execution through dedicated CUDA streams.	- Default synchronization behavior <pre> 1 '''Python 2 def forward(self, x): 3 # Each CUDA operation implicitly synchronizes 4 x = x.contiguous() 5 result = self.conv_transpose3d(x) 6 return result 7 ''' </pre>	- Minimal Synchronization <pre> 1 '''Python 2 def forward(self, x): 3 # Create dedicated stream for computation 4 with torch.cuda.stream(self.compute_stream): 5 # Operations run asynchronously in this stream 6 x_optimized = 7 x.contiguous(memory_format=torch.channels_last_3d) 8 result = self.conv_transpose3d(x_optimized) 9 # Implicit synchronization only happens when 10 result is used 11 return result 12 ''' </pre>

Table 13: (Part 4) Code snippets before and after optimizations.

Tech + Desc	Before optimization	After optimization
Thread Coarsening Thread Coarsening is an optimization technique where each thread processes multiple data elements instead of just one, increasing arithmetic intensity and reducing thread overhead.	- Each thread processes one feature element <pre> 1 '''cuda 2 for (int d = tx; d < feature_size; d += threads_x) { 3 scalar_t x_val = x[b * max_sample * feature_size 4 + n * feature_size + d]; 5 atomicAdd(&vld[k * feature_size_padded + d], 6 assign_val * x_val); 7 } 8 ''' </pre>	- Each thread processes two feature elements at once <pre> 1 '''cuda 2 #pragma unroll 4 3 for (int d = tx; d < feature_size - 1; d += 4 threads_x * 2) { 5 scalar_t x_val1 = x[b * max_sample * 6 feature_size + n * feature_size + d]; 7 scalar_t x_val2 = x[b * max_sample * 8 feature_size + n * feature_size + d + 9 threads_x]; 10 atomicAdd(&vld[k * feature_size_padded + d], 11 assign_val * x_val1); 12 atomicAdd(&vld[k * feature_size_padded + d + 13 threads_x], assign_val * x_val2); 14 } 15 // Handle remaining elements 16 for (int d = tx + (feature_size / threads_x) * 17 threads_x * 2; d < feature_size; d += 18 threads_x) { 19 scalar_t x_val = x[b * max_sample * feature_size 20 + n * feature_size + d]; 21 atomicAdd(&vld[k * feature_size_padded + d], 22 assign_val * x_val); 23 } 24 ''' </pre>
Asynchronous Execution Asynchronous Execution in CUDA allows operations to be queued and executed concurrently on separate streams, enabling overlapping computation with memory transfers for improved GPU utilization.	- Sequential execution <pre> 1 '''Python 2 def forward(self, x): 3 # Operations execute in the default stream, 4 # blocking sequentially 5 result = self.conv_transpose3d(x) 6 return result 7 ''' </pre>	- Asynchronous execution with custom stream <pre> 1 '''Python 2 def forward(self, x): 3 # Create dedicated compute stream 4 self.compute_stream = 5 torch.cuda.Stream(priority=-1) # High 6 priority stream 7 8 # Execute operations asynchronously in the 9 # custom stream 10 with torch.cuda.stream(self.compute_stream): 11 result = self._optimized_cuda_forward(x, 12 x.dtype) 13 14 # Control returns immediately while computation 15 # continues in background 16 return result 17 ''' </pre>

Table 14: (Part 5) Code snippets before and after optimizations.

B Case Study: Comparing Reference Code and CUDA-L1 Optimized Neural Network Implementations

B.1 LSTMs

Table 15: Reference code and CUDA-L1 generation for LSTM class

LSTM | Reference Code - Simple baseline implementation

```
1 import torch
2 import torch.nn as nn
3
4 class Model(nn.Module):
5     def __init__(self, input_size, hidden_size, num_layers, output_size, dropout=0.0):
6         """
7         Initialize the LSTM model.
8         """
9         super(Model, self).__init__()
10        # Initialize hidden state with random values
11        self.h0 = torch.randn((num_layers, batch_size, hidden_size))
12        self.c0 = torch.randn((num_layers, batch_size, hidden_size))
13        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True, dropout=dropout,
14                             bidirectional=False)
15        self.fc = nn.Linear(hidden_size, output_size)
16
17    def forward(self, x):
18        """
19        Forward pass through the LSTM model.
20        """
21        self.h0 = self.h0.to(x.device)
22        self.c0 = self.c0.to(x.device) # BUG: This should be self.c0.to(x.device)
23
24        # Forward propagate LSTM
25        out, state = self.lstm(x, (self.h0, self.c0)) # shape of out: (batch_size, seq_length, hidden_size)
26
27        # Decode the hidden state of the last time step
28        out = self.fc(out[:, -1, :]) # shape of out: (batch_size, output_size)
29
30        return state[0]
31
32 # Test code
33 batch_size = 10
34 sequence_length = 512
35 input_size = 128
36 hidden_size = 256
37 num_layers = 6
38 output_size = 10
39 dropout = 0.0
40
41 def get_inputs():
42     return [torch.randn(batch_size, sequence_length, input_size)]
43
44 def get_init_inputs():
45     return [input_size, hidden_size, num_layers, output_size, dropout]
```

LSTM | Fully Optimized Code - All optimizations enabled (3.4x faster)

```
1 import torch
2 import torch.nn as nn
3 import torch.cuda as cuda
4
5 class ModelNew(nn.Module):
6     def __init__(self, input_size, hidden_size, num_layers, output_size, dropout=0.0):
7         """
8         Initialize the LSTM model with three core optimization techniques.
9
10        Color coding:
```

```

11 - BLUE: CUDA Graphs optimization
12 - GREEN: Memory Contiguity optimization
13 - ORANGE: Static Tensor Reuse optimization
14 """
15 super(ModelNew, self).__init__()
16
17 # Initialize hidden states as buffers
18 self.register_buffer('h0', torch.randn((num_layers, batch_size, hidden_size)))
19 self.register_buffer('c0', torch.randn((num_layers, batch_size, hidden_size)))
20
21 # Use PyTorch's optimized LSTM implementation
22 self.lstm = nn.LSTM(
23     input_size=input_size,
24     hidden_size=hidden_size,
25     num_layers=num_layers,
26     batch_first=True,
27     dropout=dropout,
28     bidirectional=False
29 )
30
31 self.fc = nn.Linear(hidden_size, output_size)
32
33 # BLUE: CUDA GRAPHS: Variables for graph capture and replay
34 self.graph = None
35 self.graph_ready = False
36 self.input_shape = None
37
38 # ORANGE: STATIC TENSOR REUSE: Pre-allocated tensors for graph execution
39 self.static_input = None
40 self.static_output = None
41
42 # BLUE: CUDA GRAPHS: Streams for graph operations
43 self.graph_stream = None
44
45 # Track if we're running on CUDA
46 self.is_cuda_available = torch.cuda.is_available()
47
48 def _initialize_cuda_resources(self):
49     """BLUE: CUDA GRAPHS: Initialize CUDA stream for graph operations"""
50     if self.graph_stream is None:
51         self.graph_stream = cuda.Stream()
52
53 def _capture_graph(self, x, result):
54     """
55     BLUE: CUDA GRAPHS: Capture the computation graph for replay
56     ORANGE: STATIC TENSOR REUSE: Create static tensors for graph capture
57     """
58     # ORANGE: STATIC TENSOR REUSE: Clone tensors for static allocation
59     self.static_input = x.clone()
60     self.static_output = result.clone()
61
62     # BLUE: CUDA GRAPHS: Capture the computation graph
63     with torch.cuda.stream(self.graph_stream):
64         self.graph = cuda.CUDAGraph()
65         with cuda.graph(self.graph):
66             # Operations to capture in the graph
67             static_out, _ = self.lstm(self.static_input, (self.h0, self.c0))
68
69     # GREEN: MEMORY CONTIGUITY: Ensure contiguous memory layout
70     static_last = static_out[:, -1, :].contiguous()
71
72     self.static_output.copy_(self.fc(static_last))
73
74 # Wait for graph capture to complete
75 torch.cuda.synchronize()

```

```

76     # Mark graph as ready for use
77     self.graph_ready = True
78
79
80     def _standard_forward(self, x):
81         """Standard forward pass with memory contiguity optimization"""
82
83         # 🟢 MEMORY CONTIGUITY: Ensure input is contiguous
84         if not x.is_contiguous():
85             x = x.contiguous()
86
87         # Forward pass through LSTM
88         out, _ = self.lstm(x, (self.h0, self.c0))
89
90         # 🟢 MEMORY CONTIGUITY: Make last output contiguous for optimal memory access
91         last_out = out[:, -1, :].contiguous()
92
93         return self.fc(last_out)
94
95     def forward(self, x):
96         """
97         Forward pass through the LSTM model with three optimization techniques.
98
99         Optimization flow:
100         1. 🟡 CUDA GRAPHS: Check if we can use the captured graph (fast path)
101         2. 🟠 STATIC TENSOR REUSE: Use pre-allocated tensors for graph replay
102         3. 🟢 MEMORY CONTIGUITY: Ensure optimal memory layout throughout
103         """
104
105         # 🟡 CUDA GRAPHS: Fast path - use captured graph if available
106         if (x.is_cuda and self.graph_ready and x.shape == self.input_shape):
107
108             # 🟠 STATIC TENSOR REUSE: Copy to pre-allocated tensor with non-blocking transfer
109             self.static_input.copy_(x, non_blocking=True)
110
111             # 🟡 CUDA GRAPHS: Replay the captured graph
112             self.graph.replay()
113
114             # Return the output from static buffer
115             return self.static_output.clone()
116
117         # Standard execution path
118         with torch.no_grad():
119             result = self._standard_forward(x)
120
121         # 🟡 CUDA GRAPHS: Initialize graph on first CUDA input
122         if x.is_cuda and self.is_cuda_available and not self.graph_ready:
123             try:
124                 # Store the current input shape
125                 self.input_shape = x.shape
126
127                 # 🟡 CUDA GRAPHS: Initialize CUDA resources
128                 self._initialize_cuda_resources()
129
130                 # 🟡 CUDA GRAPHS + 🟠 STATIC TENSOR REUSE: Capture the graph
131                 self._capture_graph(x, result)
132
133             except Exception as e:
134                 # If graph capture fails, continue without it
135                 self.graph_ready = False
136
137         return result
138
139     # Hyperparameters from the reference implementation
140     batch_size = 10

```

```

141 sequence_length = 512
142 input_size = 128
143 hidden_size = 256
144 num_layers = 6
145 output_size = 10
146 dropout = 0.0
147
148 def get_inputs():
149     return [torch.randn(batch_size, sequence_length, input_size)]
150
151 def get_init_inputs():
152     return [input_size, hidden_size, num_layers, output_size, dropout]
153
154 # Example usage demonstrating the three techniques
155 if __name__ == "__main__":
156     import time
157
158     print("🟡 BLUE: CUDA Graphs optimization")
159     print("🟢 GREEN: Memory Contiguity optimization")
160     print("🟠 ORANGE: Static Tensor Reuse optimization")
161     print("==" * 60)
162
163     # Create model
164     model = ModelNew(*get_init_inputs())
165     model.eval()
166
167     # Test input
168     x = get_inputs()[0]
169
170     # Move to GPU if available
171     if torch.cuda.is_available():
172         model = model.cuda()
173         x = x.cuda()
174
175     print("Running on CUDA - all three optimizations active")
176
177     # First run - captures graph
178     print("\n🟡 First forward pass: Capturing CUDA graph...")
179     with torch.no_grad():
180         output = model(x)
181     print(f"    Output shape: {output.shape}")
182     print(f"    Graph ready: {model.graph_ready}")
183
184     # Subsequent runs - uses captured graph
185     print("\n🟡 Subsequent passes: Using captured graph with")
186     print("🟠 static tensor reuse and 🟢 memory contiguity")
187
188     # Warmup
189     for _ in range(10):
190         with torch.no_grad():
191             _ = model(x)
192
193     # Measure performance
194     torch.cuda.synchronize()
195     start_event = torch.cuda.Event(enable_timing=True)
196     end_event = torch.cuda.Event(enable_timing=True)
197
198     n_runs = 100
199     start_event.record()
200     with torch.no_grad():
201         for _ in range(n_runs):
202             output = model(x)
203     end_event.record()
204
205     torch.cuda.synchronize()
206     avg_time = start_event.elapsed_time(end_event) / n_runs
207

```

```
208     print(f"\nPerformance: {avg_time:.3f} ms per forward pass")
209     print(f"    Expected speedup: ~3.42x with all optimizations")
210
211 else:
212     print("\n⚠ Running on CPU - only 🟢 memory contiguity active")
213     print("    (CUDA graphs and static tensor reuse require GPU)")
214
215     with torch.no_grad():
216         output = model(x)
217     print(f"\n    Output shape: {output.shape}")
```

B.2 3DConv

Table 16: Reference code and CUDA-L1 generation for Conv3D class

Conv3D | Reference Code - Simple baseline implementation

```
1 import torch
2 import torch.nn as nn
3
4 class Model(nn.Module):
5     """
6     Model that performs a 3D convolution, applies Group Normalization, minimum, clamp, and dropout.
7     """
8     def __init__(self, in_channels, out_channels, kernel_size, groups, min_value, max_value, dropout_p):
9         super(Model, self).__init__()
10         self.conv = nn.Conv3d(in_channels, out_channels, kernel_size)
11         self.norm = nn.GroupNorm(groups, out_channels)
12         self.dropout = nn.Dropout(dropout_p)
13         self.min_value = min_value
14         self.max_value = max_value
15
16     def forward(self, x):
17         x = self.conv(x)
18         x = self.norm(x)
19         x = torch.min(x, torch.tensor(self.min_value))
20         x = torch.clamp(x, min=self.min_value, max=self.max_value)
21         x = self.dropout(x)
22         return x
23
24 # Hyperparameters
25 batch_size = 128
26 in_channels = 3
27 out_channels = 16
28 depth, height, width = 16, 32, 32
29 kernel_size = 3
30 groups = 8
31 min_value = 0.0
32 max_value = 1.0
33 dropout_p = 0.2
34
35 def get_inputs():
36     return [torch.randn(batch_size, in_channels, depth, height, width)]
37
38 def get_init_inputs():
39     return [in_channels, out_channels, kernel_size, groups, min_value, max_value, dropout_p]
```

Conv3D | Fully Optimized Code - All optimizations enabled (120x faster)

```
1 import torch
2 import torch.nn as nn
3
4 # Hyperparameters
5 batch_size = 128
6 in_channels = 3
7 out_channels = 16
8 depth, height, width = 16, 32, 32
9 kernel_size = 3
10 groups = 8
11 min_value = 0.0
12 max_value = 1.0
13 dropout_p = 0.2
14
15 class ModelNew(nn.Module):
16     def __init__(self, in_channels, out_channels, kernel_size, groups, min_value, max_value, dropout_p):
17         super(ModelNew, self).__init__()
18         # Store the original layers for parameter compatibility
19         self.conv = nn.Conv3d(in_channels, out_channels, kernel_size)
20         self.norm = nn.GroupNorm(groups, out_channels)
```

```

21     self.dropout = nn.Dropout(dropout_p)
22     self.min_value = min_value
23     self.max_value = max_value
24     self.dropout_p = dropout_p
25
26     # 🟦 TECH 1: Mathematical Short-Circuit Optimization
27     # Detects when min_value=0.0 to skip entire computation
28     self.use_optimized_path = (min_value == 0.0)
29
30     # 🟡 TECH 4: Pre-computed Convolution Parameters
31     # Extract and store conv parameters once during initialization
32     if isinstance(kernel_size, int):
33         self.kernel_size = (kernel_size, kernel_size, kernel_size)
34     else:
35         self.kernel_size = kernel_size
36     self.stride = self.conv.stride
37     self.padding = self.conv.padding
38     self.dilation = self.conv.dilation
39
40     # 🟡 TECH 4: Pre-compute output dimensions for standard input
41     self.out_depth = ((depth + 2 * self.padding[0] - self.dilation[0] * (self.kernel_size[0] - 1) - 1)
42                       // self.stride[0]) + 1
43     self.out_height = ((height + 2 * self.padding[1] - self.dilation[1] * (self.kernel_size[1] - 1) - 1)
44                       // self.stride[1]) + 1
45     self.out_width = ((width + 2 * self.padding[2] - self.dilation[2] * (self.kernel_size[2] - 1) - 1)
46                      // self.stride[2]) + 1
47
48     # Standard output shape for the default batch size
49     self.standard_shape = (batch_size, out_channels, self.out_depth, self.out_height, self.out_width)
50
51     # 🟣 TECH 2: Pre-allocated Zero Tensors
52     # Create zero tensors once to avoid allocation overhead
53     if self.use_optimized_path:
54         self.register_buffer('zero_output_float32',
55                             torch.zeros(self.standard_shape, dtype=torch.float32),
56                             persistent=False)
57         self.register_buffer('zero_output_float16',
58                             torch.zeros(self.standard_shape, dtype=torch.float16),
59                             persistent=False)
60         self.register_buffer('zero_output_bfloat16',
61                             torch.zeros(self.standard_shape, dtype=torch.bfloat16),
62                             persistent=False)
63
64     def calculate_output_shape(self, input_shape):
65         """Calculate the output shape of the convolution operation."""
66         batch_size, _, d, h, w = input_shape
67
68         # 🟡 TECH 4: Use precomputed parameters
69         # Avoid repeated attribute lookups
70         out_d = ((d + 2 * self.padding[0] - self.dilation[0] * (self.kernel_size[0] - 1) - 1) //
71                 self.stride[0]) + 1
72         out_h = ((h + 2 * self.padding[1] - self.dilation[1] * (self.kernel_size[1] - 1) - 1) //
73                 self.stride[1]) + 1
74         out_w = ((w + 2 * self.padding[2] - self.dilation[2] * (self.kernel_size[2] - 1) - 1) //
75                 self.stride[2]) + 1
76
77         return (batch_size, self.conv.out_channels, out_d, out_h, out_w)
78
79     def forward(self, x):
80         # 🟦 TECH 1: Mathematical Short-Circuit - Main optimization
81         # Skip all computation when we know result will be zeros
82         if not self.use_optimized_path:
83             # Standard path for non-optimized cases
84             x = self.conv(x)
85             x = self.norm(x)
86             x = torch.minimum(x, torch.tensor(self.min_value, device=x.device))
87             x = torch.clamp(x, min=self.min_value, max=self.max_value)

```

```

82         x = self.dropout(x)
83         return x
84
85     # Optimized path when min_value == 0.0
86     # Since min(x, 0) followed by clamp(0, 1) always produces zeros
87
88     # 🟢 TECH 3: Direct Shape Matching
89     # Fast path for standard input dimensions
90     if x.shape == (batch_size, in_channels, depth, height, width):
91
92         # 🟡 TECH 2: Use pre-allocated tensors
93         # Return pre-allocated zeros matching input dtype
94         if x.dtype == torch.float32:
95             return self.zero_output_float32
96         elif x.dtype == torch.float16:
97             return self.zero_output_float16
98         elif x.dtype == torch.bfloat16:
99             return self.zero_output_bfloat16
100        else:
101            # Fallback for other dtypes
102            return torch.zeros(self.standard_shape, device=x.device, dtype=x.dtype)
103    else:
104        # For non-standard input shapes, calculate output shape
105        output_shape = self.calculate_output_shape(x.shape)
106        return torch.zeros(output_shape, device=x.device, dtype=x.dtype)
107
108    def get_inputs():
109        return [torch.randn(batch_size, in_channels, depth, height, width)]
110
111    def get_init_inputs():
112        return [in_channels, out_channels, kernel_size, groups, min_value, max_value, dropout_p]
113
114    # Color Legend:
115    # 🟡 TECH 1: Mathematical Short-Circuit (Blue) - Skips computation when min_value=0
116    # 🟢 TECH 2: Pre-allocated Tensors (Purple) - Pre-allocates zero tensors
117    # 🟢 TECH 3: Direct Shape Matching (Green) - Fast path for standard shapes
118    # 🟡 TECH 4: Pre-computed Parameters (Orange) - Pre-computes conv parameters

```

References

- [1] The expectation-maximization algorithm. *IEEE Signal processing magazine* 13, 6 (1996), 47–60.
- [2] BÄCK, T., AND SCHWEFEL, H.-P. An overview of evolutionary algorithms for parameter optimization. *Evolutionary computation* 1, 1 (1993), 1–23.
- [3] BLEI, D. M., KUCUKELBIR, A., AND MCAULIFFE, J. D. Variational inference: A review for statisticians. *Journal of the American statistical Association* 112, 518 (2017), 859–877.
- [4] CHEN, W., ZHU, J., FAN, Q., MA, Y., AND ZOU, A. Cuda-llm: Llms can write efficient cuda kernels. *arXiv preprint arXiv:2506.09092* (2025).
- [5] CUMMINS, C., SEEKER, V., GRUBISIC, D., ROZIERE, B., GEHRING, J., SYNNAEVE, G., AND LEATHER, H. Llm compiler: Foundation language models for compiler optimization. In *Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction* (2025), pp. 141–153.
- [6] DAT, P. V. T., DOAN, L., AND BINH, H. T. T. Hsevo: Elevating automatic heuristic design with diversity-driven harmony search and genetic algorithm using llms. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2025), vol. 39, pp. 26931–26938.
- [7] GRATTAFIORI, A., DUBEY, A., JAUHRI, A., PANDEY, A., KADIAN, A., AL-DAHLE, A., LETMAN, A., MATHUR, A., SCHELLEN, A., VAUGHAN, A., ET AL. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [8] GUO, D., YANG, D., ZHANG, H., SONG, J., ZHANG, R., XU, R., ZHU, Q., MA, S., WANG, P., BI, X., ET AL. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [9] HURST, A., LERER, A., GOUCHER, A. P., PERELMAN, A., RAMESH, A., CLARK, A., OSTROW, A., WELIHINDA, A., HAYES, A., RADFORD, A., ET AL. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).
- [10] JAECH, A., KALAI, A., LERER, A., RICHARDSON, A., EL-KISHKY, A., LOW, A., HELYAR, A., MADRY, A., BEUTEL, A., CARNEY, A., ET AL. Openai o1 system card. *arXiv preprint arXiv:2412.16720* (2024).
- [11] JIANG, A. Q., SABLAYROLLES, A., ROUX, A., MENSCH, A., SAVARY, B., BAMFORD, C., CHAPLOT, D. S., CASAS, D. D. L., HANNA, E. B., BRESSAND, F., ET AL. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).
- [12] KONDA, V., AND TSITSIKLIS, J. Actor-critic algorithms. *Advances in neural information processing systems* 12 (1999).
- [13] LANGE, R. T., PRASAD, A., SUN, Q., FALDOR, M., TANG, Y., AND HA, D. The ai cuda engineer: Agentic cuda kernel discovery, optimization and composition. Tech. rep., 2025.
- [14] LEE, K.-H., FISCHER, I., WU, Y.-H., MARWOOD, D., BALUJA, S., SCHUURMANS, D., AND CHEN, X. Evolving deeper llm thinking. *arXiv preprint arXiv:2501.09891* (2025).
- [15] LIU, A., FENG, B., XUE, B., WANG, B., WU, B., LU, C., ZHAO, C., DENG, C., ZHANG, C., RUAN, C., ET AL. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [16] LIU, F., TONG, X., YUAN, M., LIN, X., LUO, F., WANG, Z., LU, Z., AND ZHANG, Q. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. *arXiv preprint arXiv:2401.02051* (2024).
- [17] MUENNIGHOFF, N., YANG, Z., SHI, W., LI, X. L., LI, F.-F., HAJISHIRZI, H., ZETTLEMOYER, L. S., LIANG, P., CANDES, E. J., AND HASHIMOTO, T. sl: Simple test-time scaling. *ArXiv abs/2501.19393* (2025).
- [18] NOVIKOV, A., VŮ, N., EISENBERGER, M., DUPONT, E., HUANG, P.-S., WAGNER, A. Z., SHIROBOKOV, S., KOZLOVSKII, B., RUIZ, F. J., MEHRABIAN, A., ET AL. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131* (2025).
- [19] OLMo, T., WALSH, P., SOLDAINI, L., GROENEVELD, D., LO, K., ARORA, S., BHAGIA, A., GU, Y., HUANG, S., JORDAN, M., ET AL. 2 olmo 2 furious. *arXiv preprint arXiv:2501.00656* (2024).
- [20] OUYANG, A., GUO, S., ARORA, S., ZHANG, A. L., HU, W., RÉ, C., AND MIRHOSEINI, A. Kernelbench: Can llms write efficient gpu kernels? *arXiv preprint arXiv:2502.10517* (2025).
- [21] ROMERA-PAREDES, B., BAREKATAIN, M., NOVIKOV, A., BALOG, M., KUMAR, M. P., DUPONT, E., RUIZ, F. J., ELLENBERG, J. S., WANG, P., FAWZI, O., ET AL. Mathematical discoveries from program search with large language models. *Nature* 625, 7995 (2024), 468–475.
- [22] SCHULMAN, J., WOLSKI, F., DHARIWAL, P., RADFORD, A., AND KLIMOV, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [23] SHAO, Z., WANG, P., ZHU, Q., XU, R., SONG, J., BI, X., ZHANG, H., ZHANG, M., LI, Y., WU, Y., ET AL. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).
- [24] SHENGYU, Z., LINFENG, D., XIAOYA, L., SEN, Z., XIAOFEI, S., SHUHE, W., JIWEI, L., HU, R., TIANWEI, Z., WU, F., ET AL. Instruction tuning for large language models: A survey. *arXiv preprint arXiv:2308.10792* (2023).
- [25] TEAM, G., ANIL, R., BORGEAUD, S., ALAYRAC, J.-B., YU, J., SORICUT, R., SCHALKWYK, J., DAI, A. M., HAUTH, A., MILLICAN, K., ET AL. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).

- [26] TEAM, G., MESNARD, T., HARDIN, C., DADASHI, R., BHUPATIRAJU, S., PATHAK, S., SIFRE, L., RIVIÈRE, M., KALE, M. S., LOVE, J., ET AL. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295* (2024).
- [27] WANG, S., ZHANG, S., ZHANG, J., HU, R., LI, X., ZHANG, T., LI, J., WU, F., WANG, G., AND HOVY, E. Reinforcement learning enhanced llms: A survey. *arXiv preprint arXiv:2412.10400* (2024).
- [28] WEI, A., SURESH, T., TAN, H., XU, Y., SINGH, G., WANG, K., AND AIKEN, A. Improving assembly code performance with large language models via reinforcement learning. *arXiv preprint arXiv:2505.11480* (2025).
- [29] WEI, J., WANG, X., SCHUURMANS, D., BOSMA, M., XIA, F., CHI, E., LE, Q. V., ZHOU, D., ET AL. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [30] WEI, Y., DUCHENNE, O., COPET, J., CARBONNEAUX, Q., ZHANG, L., FRIED, D., SYNNAEVE, G., SINGH, R., AND WANG, S. I. Swe-rl: Advancing llm reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449* (2025).
- [31] WILLIAMS, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning* 8 (1992), 229–256.
- [32] YANG, A., LI, A., YANG, B., ZHANG, B., HUI, B., ZHENG, B., YU, B., GAO, C., HUANG, C., LV, C., ET AL. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [33] ZHANG, R., LIU, F., LIN, X., WANG, Z., LU, Z., AND ZHANG, Q. Understanding the importance of evolutionary search in automated heuristic design with large language models. In *International Conference on Parallel Problem Solving from Nature* (2024), Springer, pp. 185–202.